

Towards Practical and Round-Optimal (Verifiable) Oblivious Pseudorandom Functions with Post-Quantum Security and Their Applications

Abstract—The Oblivious Pseudorandom Function (OPRF) is a fundamental building block for various useful protocols, including Password-Protected Secret Sharing (PPSS), End-to-Same-End Encryption (E2SE), and Private Set Intersection (PSI). PPSS enables a user to distribute their secret information among n servers, such that the compromise of any t servers leaks no information on the secret or password. By contacting $t+1$ servers with the correct password, the secret can be reconstructed. E2SE allows any application to assist end-users in storing and retrieving private data based solely on passwords. PSI allows two parties to compute the intersection of their private sets, and underlies deployed services such as compromised-credential checking.

Despite their utility, practical and round-optimal OPRF constructions from post-quantum assumptions have yet to be developed. In anticipation of the quantum computing era, we propose the first practical two-round OPRF with malicious security based on lattice-based assumptions and symmetric key primitives that are believed to be post-quantum secure. Our novel practical OPRF has a client-side processing time of 9.3ms, and a server-side processing time of 8.5ms, which is 1700 times better than existing construction from lattice [2] on the server-side. Our key observation is that a malicious-secure OPRF can be constructed from a malicious-secure Oblivious Transfer protocol from any IND-CPA secure key encapsulation mechanism and a semi-honest secure Garbled Circuit with high efficiency. Counter-intuitively, the malicious security of the Garbled Circuit is unnecessary.

We then upgrade our OPRF to a *verifiable* OPRF (VOPRF) with post-quantum security, in which the client is guaranteed that every accepted evaluation is computed under a single committed server key. The server commits to its AES key once and for all via a small set of ciphertexts together with a STARK proof of key knowledge, yielding a reusable and publicly verifiable key commitment; for each batch of evaluations, the server additionally produces a STARK proof that the garbled circuits were correctly generated with respect to the committed key. The proof covers only the garbling step and is independent of the client's inputs, hence it can be entirely precomputed offline; the online phase remains two-round, and the client's amortized online cost approaches that of the base OPRF. In contrast to sampling-based verification paradigms, our VOPRF achieves negligible soundness for worst-case inputs.

Based on our proposed practical OPRF, we can further design efficient PPSS and E2SE protocols, and based on the proposed VOPRF, we obtain post-quantum verifiable PSI protocols in the

unbalanced setting, with password breach checking as a typical application. Our work contributes to the development of post-quantum secure cryptographic primitives, which are essential for securing sensitive information in the quantum computing era.

I. INTRODUCTION

Oblivious Pseudorandom Function (OPRF) is a crucial component in various protocols, including Password-Protected Secret Sharing (PPSS), End-to-Same-End Encryption (E2SE), and Private Set Intersection (PSI). PPSS allows a user to distribute their secret information among n servers, such that the compromise of any t servers reveals no information on the secret or password. By contacting $t+1$ servers with the correct password, the secret can be reconstructed. E2SE enables users to store and retrieve private data securely based solely on passwords. PSI allows two parties to compute the intersection of their private sets without revealing anything else; in the unbalanced setting, where a client holds a few items and a server holds a large set, PSI is the core of deployed compromised-credential checking services.

Despite their utility, practical and round-optimal OPRF constructions from post-quantum assumptions have not yet been developed. In anticipation of the quantum computing era, we propose the first practical two-round OPRF with malicious security based on lattice-based assumptions and symmetric key primitives that are believed to be post-quantum secure. Our novel OPRF has a client-side processing time of 9.3ms and a server-side processing time of 8.5ms. Our key observation is that a malicious-secure OPRF can be constructed from a malicious-secure Oblivious Transfer protocol using any IND-CPA secure key encapsulation mechanism and a semi-honest secure Garbled Circuit with high efficiency. Counterintuitively, the malicious security of the Garbled Circuit is unnecessary.

We further upgrade our OPRF to a verifiable OPRF (VOPRF) with post-quantum security, in which the client is guaranteed that every accepted evaluation is computed under a single committed server key. Following a two-layer design, the server commits to its AES key once via a small set of ciphertexts together with a STARK proof of key knowledge (a reusable, publicly verifiable key commitment), and proves in one STARK per batch that the garbled circuits were correctly generated with respect to the committed key. The proof covers only the garbling step and is independent of the client's inputs, hence it can be entirely precomputed offline; the online phase remains two-round and the client's amortized online cost approaches that of the base OPRF.

Based on our proposed practical OPRF, we can design efficient PPSS and E2SE protocols, and based on the proposed

VOPRF, we obtain post-quantum verifiable PSI protocols in the unbalanced setting, with password breach checking as a typical application. Our work contributes to the development of post-quantum secure cryptographic primitives, which are essential for securing sensitive information in the quantum computing era.

One key building block of password-protected secure storage (PPSS) and end-to-end encryption (E2SE) is oblivious pseudorandom function (OPRF). An OPRF is a two-party protocol in which a client and a server jointly evaluate a pseudorandom function (PRF) $f_k(\cdot)$ on an input x , where the client selects x and the server holds the key k . An OPRF ensures that a malicious server does not gain knowledge of the input x , or the output $y := f_k(x)$, while a malicious client only learns the value of $f_k(\cdot)$ at the query point x . The protocol's malicious security allows both parties to arbitrarily violate it.

In password-based schemes like PPSS and E2SE, the OPRF protocol provides clients with high-entropy randomness that binds to their specific low-entropy password, thereby offering resistance against offline dictionary attacks. Additionally, when using the same password and key, a multi-point OPRF can provide the client with identical randomness for authentication (while this is not feasible with a single-point OPRF). OPRFs are multi-point if with one randomly chosen key, the evaluation points can be adaptively chosen as a standard PRF evaluation (instead, in a single-point OPRF, the two parties jointly generate a random key, as well as the evaluation, on each execution). An OPRF is round optimal if it only needs two messages, which can significantly decrease the communication latency of PPSS and E2SE schemes.¹

However, most existing OPRF constructions are also based on classical assumptions such as decisional Diffie-Hellman (DDH) and RSA and their variants, which are known to be vulnerable to quantum attacks. Therefore, finding an appropriate quantum secure OPRF in the malicious setting is crucial to design post-quantum secure PPSS and E2SE. Unfortunately, the currently known candidates are not yet efficient, particularly in the round-optimal case.²

Recently, several round-optimal Oblivious Pseudo-Random Function (OPRF) constructions based on lattice assumptions have been proposed [2], [3]. However, despite their theoretical security, these constructions are currently impractical. Albrecht et al. [3] introduced the first round-optimal post-quantum OPRF that maintains security based on sub-exponential large modulus lattice assumptions. Unfortunately, these assumptions are weak, thus require the concrete parameters to be too large to be useful, rendering the construction purely theoretical, even in the semi-honest setting. Then, Albrecht et al. [2] proposed a novel partially OPRF (POPRF) from shallow pseudorandom functions [12] and Torus Fully Homomorphic Encryption (TFHE) [21] based on standard Ring-LWE assumptions. The semi-honest version of this construction can be implemented

within seconds. However, achieving malicious security for these OPRFs still uses heavy post-quantum (general) non-interactive zero-knowledge (NIZK) protocols [11], [57], [60], which are not yet known to be efficiently instantiable.³ For detailed comparisons, see Table I.

More recently, several practical post-quantum OPRFs have emerged. Beullens et al. [10] proposed the 2Hash framework that compiles secure evaluation of Legendre- or AES-based functions into UC-secure OPRFs, whose efficient instantiation requires nine rounds and about 911KB of communication. Yang et al. [61] constructed batched OPRFs from the power-residue PRF and VOLE with attractive amortized communication, whose malicious-secure variant requires at least five rounds, and whose security holds against quantum adversaries with classical queries only. Esgin et al. [26] gave the round-optimal verifiable OPRF LeOPaRd from new interactive lattice assumptions, whose communication is estimated from proof systems without an implementation. Heimberger et al. [34] proposed Leap, a fast lattice-based OPRF with an unbalanced PSI application, which is semi-honest secure and not verifiable. Closest to our approach, Faller et al. [28] revisited OPRFs from garbled circuits in the UC model; their base protocol is passively secure, and verifiability is obtained by checking a key commitment inside the circuit, which enlarges the evaluated circuit and is proved per evaluation. In contrast, our VOPRF keeps the evaluated circuit intact, proves correct garbling for a whole batch at once, and the resulting proof is publicly verifiable and precomputable offline.

A. Our Contributions

In this work, we aim to construct *practical* round-optimal OPRFs from post-quantum computational assumptions with both security against malicious client and server, which is *necessary* for PPSS, and E2SE. We will then investigate how to build practical PPSS and E2SE schemes from these OPRFs.

Intuitively, the design of OPRF can be viewed as a special case of two-party secure computation. We observe that MPC-style malicious security is unnecessary for many applications, such as PPSS and E2SE. Specifically, we observe that ensuring the a public verifiability (which was one root reason of using general NIZK proofs) of the PRF evaluation may not be necessary. Instead, it suffices to guarantee that the output received by the client is pseudorandom in the view of the server. In this way, we can get rid of the full-fledged NIZK. More importantly, this seemingly weakened OPRF still suffices for the more complicated applications of PPSS and E2SE. We therefore propose a new approach to designing OPRF by adopting Oblivious Transfer (OT) (with malicious security) and Garbled Circuits (GC) (with semi-honest security) to compute the AES encryption function. We provide a rigorous proof of our new generic OPRF framework under the game-based model proposed by Chen et al. [20]. The critical aspect is to ensure the evaluation is still pseudorandom even to a malicious server who does not need to prove its honest behaviors.

To demonstrate the efficiency of our approach, we instantiate the Oblivious Transfer (OT) scheme from the post-quantum

¹We would also stress that malicious security of OPRF is crucial for those applications like PPSS, and E2SE, otherwise trivializing the problems.

²It is worth noting that some private set intersection (PSI) protocols [18], [41], [48], [49], [51] are constructed using batched *single-point* OPRFs based on Vector Oblivious Linear Evaluation (VOLE) [14], which could be efficient. However, the keys of these single-point OPRFs are randomly chosen for each execution and cannot be reused to evaluate two different inputs, which is a necessary property for most applications, such as password-based protocols.

³The current post-quantum interactive zero-knowledge constructions [8], [59] may be much more efficient than NIZKs, but these would significantly increase the number of communication rounds.

TABLE I. POST-QUANTUM (V)OPRFs CANDIDATES IN THE LITERATURE

Work	Assumptions	Rounds	Communication Cost	ROM/QROM	Trusted Setup	Malicious	Verifiable	Implementation
[3]	RLWE and SIS	2	$\approx 2\text{MB}$	QROM	$\times$	$\times$	$\times$	$\times$
[3]	RLWE and SIS	2	$>128\text{GB}$	QROM	$\times$	$\checkmark$	$\checkmark$	$\times$
[54]	Legendre PRF	3	$\approx \lambda \cdot 13\text{K}$	ROM	$\triangle^1$	$\times$	$\times$	$\times$
[13]	CSIDH	3	424KB	/	$\times$	$\ast^2$	$\times$	$\times$
[6]	SIDH	2	3.0MB	ROM	$\checkmark$	$\checkmark$	$\checkmark$	$\times$
[35]	CSIDH	2	21KB	/	$\checkmark$	$\times$	$\times$	$\times$
[35]	CSIDH	4	35KB	/	$\checkmark$	$\ast$	$\times$	$\times$
[35]	CSIDH	258	25KB	/	$\times$	$\times$	$\times$	$\times$
[25]	[12]	2	80B	/	$\triangle$	$\times$	$\times$	$\times$
[2]	[12], lattices	2	16.9MB	ROM	$\times$	$\ast$	$\times$	$\checkmark^3$
[2]	[12], lattices	2	4078.5MB	ROM	$\times$	$\checkmark$	$\checkmark$	$\times$
[28]	AES, OT (UC)	2	$\approx 0.2\text{MB}$	ROM	$\times$	$\times^4$	$\checkmark^4$	$\checkmark$
[10]	Legendre PRF, VOLE-ZK	9	$\approx 911\text{KB}$	ROM	$\times$	$\checkmark$	$\checkmark$	$\checkmark$
[61]	Power-residue PRF ⁵ , VOLE	≥ 5	$\geq 1.6\text{MB}^6$	ROM	$\times$	$\checkmark$	$\checkmark$	$\checkmark$
[26]	lattices (interactive)	2	$\approx 137\text{KB}^7$	ROM	$\times$	$\checkmark$	$\checkmark$	$\times$
[34]	LWR, OT and OLE	2	23KB online	ROM	$\triangle$	$\times$	$\times$	$\checkmark$
Ours (OPRF)	AES ⁸ , lattices ⁹	2	0.2MB	ROM	$\times$	$\checkmark$	$\times$	$\checkmark$
Ours (VOPRF)	AES ⁸ , lattices ⁹ , STARK	2	0.2MB + amortized proof ¹⁰	ROM	$\times$	$\checkmark$	$\checkmark^{11}$	$\checkmark$

¹ " $\triangle$ " denotes the protocol do not rely on trusted setups but a preprocessing phase to share correlated randomness.

² " $\ast$ " denotes the security model with only malicious clients but semi-honest servers.

³ The implementation does not include the zero-knowledge arguments, so we emphasize that the security model of implementation remains semi-honest.

⁴ The base protocol of [28] is passively secure in the UC model; verifiability is obtained by checking a commitment to the server key inside the circuit, which enlarges the evaluated circuit.

⁵ The power-residue PRF admits efficient quantum distinguishers under quantum queries; [61] restricts to quantum adversaries with classical queries.

⁶ Total communication without amortization as reported in [26]; the amortized online communication of their batched malicious protocol is about 1.9KB.

⁷ Estimated from LaBRADOR proof sizes; no implementation is provided in [26].

⁸ We assume that the family of fixed uniformly chosen random keys used in the AES encryption function is a PRF family.

⁹ More precisely, our OPRF is based on AES and OTs, and our OTs are instantiated by lattice-based OTs in the implementation, it can be also instantiated with other post-quantum computational assumptions such as isogeny and LPN.

¹⁰ The STARK proof is generated offline and its cost is amortized over a batch of evaluations; see Section V for details.

¹¹ Our VOPRF is publicly verifiable: anyone holding the key commitment can verify the proof.

key encapsulation mechanisms (KEMs) and implement the protocol on a personal computer (PC). Our OPRF is based on assumptions and primitives with can resist quantum attacks. Client-side processing time is 9.3ms, and server-side processing time is 8.5ms. Compared to the post-quantum torus fully homomorphic encryption (TFHE)-based OPRF with semi-honest OPRF, our protocol is significantly faster ($30\times$ for client, while $1700\times$ for server). The TFHE-based OPRF requires 256ms for clients and over 14 seconds for servers. Notably, our proposed protocol achieves performance levels similar to the widely used 2HashDH-based OPRF scheme [36], with both client-side and server-side processing times taking place within 1ms. But, our proposed protocol offers post-quantum resistance compared to the 2HashDH-based OPRF.

We then upgrade our OPRF to a verifiable OPRF (VOPRF) with post-quantum security, in which the client is guaranteed that every accepted evaluation is computed under a single committed server key. Our construction follows a two-layer design. In the key-binding layer, the server commits to its AES key once and for all by publishing a small set of AES ciphertexts together with a STARK proof of key knowledge; this yields a reusable and publicly verifiable key commitment. In the circuit-authentication layer, the server proves in a single STARK per batch that all garbled circuits were correctly generated from the AES circuit and the committed key. Two design insights make this practical: the proof need not cover the OT protocol, since malformed input labels only cause the evaluation to abort rather than to output a wrong value; and both garbling and proving are independent of the client's input, hence can be precomputed offline in a fully parallelizable manner. The online phase remains two-round, the client's amortized online cost approaches that of the base OPRF,

and the soundness error is negligible for worst-case inputs, in contrast to sampling-based verification paradigms whose soundness error is inherently inversely proportional to the amortization factor.

We then present the first practical implementation of PPSS based on our novel OPRF, which only uses assumptions and primitives that can resist quantum attacks. In both the initialization phase and the reconstruction of the (2,3)-PPSS scheme, the average client-side time cost (total time divided by 3) is 9.4ms, while the average individual server-side time cost is 8.5ms. We observe that in the PPSS scheme, the OPRF accounts for more than 90% of the time overhead, dominating the overall time cost.

Finally, we present a post-quantum verifiable PSI protocol in the unbalanced setting from our VOPRF, with password breach (compromised-credential) checking as a typical application: a client holding a few credentials obliviously learns whether they appear in the server's large database, with the guarantee that the server answered all queries under its committed key. To the best of our knowledge, this is the first post-quantum VOPRF that is simultaneously round-optimal, implemented, and publicly verifiable.

B. Technical Overview

One approach to designing Oblivious Pseudorandom Functions (OPRF) is to consider it as a maliciously secure two-party computation (2PC) for PRF between a client and a server. A simple method for this design is to combine maliciously secure Oblivious Transfer (OT) and authenticated garbling, as proposed in [58], to evaluate a standard PRF such as AES.

To verify the correctness of the evaluation result of the authenticated garbled circuit, a preprocessing phase is necessary to establish correlated randomness between the two parties, along with their corresponding information-theoretic message authentication codes (IT-MACs) [47] for each gate, following the topological order of the circuit. These randomness and IT-MACs will be used to verify the evaluation correctness for each gate during the online phase. However, the authenticated garbled circuit is much less efficient than Yao’s semi-honest garbled-circuit protocol [62]. According to experimental results presented in [58], the authenticated garbled circuit costs more than eight times as much as the semi-honest garbled circuit, not to mention the additional communication rounds and transformation data.

As previously mentioned, achieving malicious security in the evaluation of OPRF requires satisfying two main objectives: privacy against a malicious adversary and verifiability. The first objective necessitates that even if the adversary does not comply with the protocol, it must not be able to obtain any information about the honest party’s private inputs. The second objective requires that even if the adversary does not comply with the protocol, the honest party must still be able to verify the correctness of its own outputs.

However, we observe that the verifiability in OPRF is not necessarily required in many password-based secure storages, such as PPSS and E2SE. In these schemes, the honest client is not concerned about the specific value of the random output, but rather focuses on keeping the password and this output confidential from adversaries. This is because an incorrect evaluation result may only cause authentication failure, but will not cause the leakage of the secret data.

In Section III, we adapt the definition of Oblivious Pseudorandom Functions (OPRF) from [20]. Unlike the previous definitions presented in [2], [37], [56], this definition captures the property of pseudorandomness, which ensures that malicious clients who did not corrupt the server but can arbitrarily query it cannot predict the evaluation result. Additionally, the definition captures the property of obliviousness, which ensures that a malicious server cannot predict the evaluation result of the client for an unknown input x . The game is one-shot; since our client never aborts and hence the adversary obtains no feedback from the client’s computation (see Remark 2), security across multiple interactions with the same input follows from the receiver privacy of the OT via a standard hybrid argument. However, we do not require the client to verify that the evaluation result y equals $f(x)$. To show usefulness of the new definition, we prove that OPRF with the new definition still yields a PPSS (previous such relation was only known when OPRF is defined in the UC model).

Using the aforementioned definition of OPRF, we construct an OPRF from maliciously secure Oblivious Transfer (OT) and semi-honest Garbled Circuit (GC) protocols. The client, in possession of the input x , obtains the corresponding input label of x from a series of OT protocols against malicious adversaries. Subsequently, the client obliviously computes $\text{AES}(k, x)$ (more precisely, in our subsequent formal construction, it is $\text{AES}(k, H_1(x))$) by evaluating a garbled AES circuit generated by the sender, who holds the secret key k .

To still ensure strong enough security without forcing a

malicious server to prove its correct executions, we carefully analyze the important properties.

The property of pseudorandomness in our construction is based on the intuition that the security provided by Yao’s Garbled Circuit (GC) is stronger than commonly perceived. Although GC does not provide verifiability (due to the absence of authentication for a maliciously garbled circuit), it still ensures garbler privacy even against a malicious evaluator. This is because GC can always be simulated from the input and the output of the evaluation. Consequently, the evaluator cannot learn any information beyond the input and output, regardless of whether they follow the protocol. In our construction, the generation of garbled circuits and the input labels corresponding to the secret key k remain independent of the evaluator’s (client’s) behavior. Therefore, a malicious client will not learn more about the AES key except the ciphertext if they only learn the label corresponding to their inputs. We solve this problem by let the challenger

Achieving the property of obliviousness requires of addressing the threat posed by a malicious server’s ability to manipulate the client’s output. To mitigate this threat and ensure that the server cannot predict the client’s output, we utilize a random oracle in the protocol. In our construction, the client outputs $H_2(x, \text{AES}(k, x))$ instead of directly outputting the evaluation $\text{AES}(k, x)$. While this approach does not guarantee that the evaluation result is computed as per the protocol, it is sufficient to prevent the server from predicting the output if it does not know the secret input x .

We formally demonstrate above properties and give rigorous security proofs.

Upgrading to a verifiable OPRF. While forgoing verifiability suffices for PPSS and E2SE, applications such as PSI require the client to be convinced that every accepted evaluation is computed under a single committed server key. A natural idea is to verify the evaluation by *sampling*: the client mixes its real inputs with check points for which the correct outputs are known, and accepts if all check answers are correct. Unfortunately, black-box sampling can only constrain the evaluated function on the tested points: a malicious server that garbles circuits computing a function that agrees with $\text{AES}(k, \cdot)$ on the check points but errs elsewhere passes all checks while corrupting every real evaluation, and making the check points secret or distributing them as the real inputs does not rescue the soundness beyond a small constant. We therefore authenticate the *computation* itself rather than its input-output behavior on sample points.

Our VOPRF follows a two-layer design. In the *key-binding layer*, the server commits to its AES key k once and for all by publishing a small set of ciphertexts $\{\text{AES}(k, i)\}$ together with a STARK proof [9] of knowledge of a single key k consistent with all ciphertexts and with a hash commitment cm of k . Since a few plaintext-ciphertext pairs determine an AES key uniquely, this yields a reusable and publicly verifiable key commitment. In the *circuit-authentication layer*, for each batch of m evaluations, the server garbles m copies of the AES circuit and produces one STARK proof that all garbled tables were correctly generated from the AES circuit with wire labels whose key-wire labels encode the key committed in cm . The client verifies the proof, obtains its input labels via OT, and

evaluates; by the authenticity property of garbled circuits, any input label inconsistent with the committed tables only causes the evaluation to abort rather than to output a wrong value, so the proof need not cover the OT protocol. To prevent the abort event from leaking the client's input (a selective-failure attack), the client encodes each input bit into random shares before the OTs, so that aborts are statistically independent of the input. Since garbling and proving are independent of the client's input, the server can precompute garbled circuits and proofs entirely offline in a fully parallelizable manner; the online phase remains two-round, and the client's amortized online cost approaches that of the base OPRF.

C. Related Work

The concept of OPRFs was initially introduced by Freedman et al. [29], where they utilized OT to compute the Naor-Reingold PRF [46] obliviously and constructed OPRFs for keyword search applications. The applications of OPRF include the private set intersection [33], password hardening services [27], [43], [53], password management systems [55], password-hardened encryption [42], oblivious key management services [38], strong asymmetric password authenticated key exchange [39] and end-to-end encryption [20].

The 2HashDH protocol proposed by Jarecki et al. [36] is the most widely used OPRF construction. This protocol is based on the one-more Diffie-Hellman (OMDH) assumption and can be utilized to construct Password-protected Secret Sharing (PPSS) schemes with optimal round complexity. In the original construction of 2HashDH, the OPRF evaluation can be verified via NIZK. Subsequently, Jarecki et al. [37] demonstrated that the computational and communication complexity of the PPSS protocol can be further reduced by relaxing the verifiability property of the OPRF.

Extending the line of Diffie-Hellman-based OPRFs, Tyagi et al. [56] proposed 3HashSDHI, the first partially oblivious PRF (POPRF) that does not rely on bilinear pairings. They introduced property-based security definitions for POPRFs that provide verifiability against malicious servers via an embedded NIZK proof, information-theoretic request privacy against passive servers, and pseudorandomness against malicious clients with a fine-grained one-more evaluation accounting per public tag. Their pseudorandomness is proven in the ROM under a newly introduced interactive gap assumption, whose hardness is analyzed only in the algebraic group model. Hence, while their security model provides stronger guarantees such as verifiability and perfect request privacy, their construction relies on discrete-logarithm-style assumptions and is not post-quantum secure. Our work takes the opposite direction: we weaken the security model by forgoing verifiability, in exchange for post-quantum instantiability from standard assumptions (see Remark 1 in Section III).

Efficient quantum-secure algorithms for OPRF are currently an area of limited research. The detailed comparison can be found in Table I. Apart from the lattice-based construction mentioned above, Boneh et al. proposed two post-quantum OPRFs based on isogenies [13]. One of these constructions, which is related to SIDH, was found to be insecure due to a recent attack on supersingular isogeny systems [7]. The other 3-round OPRF is associated with CSIDH. Recently, a

fixed-and-improved SIDH-based candidate was proposed in [6], while an improved CSIDH-based candidate is presented in [35]. However, the concrete efficiency of isogenies-based OPRF constructions has not yet been evaluated with real experiments.

An alternative approach to constructing OPRFs is to evaluate PRFs via general MPC. In [25], an MPC-based OPRF is proposed that is secure against semi-honest adversaries. This approach builds on [12] and achieves a much smaller communication complexity compared to other post-quantum candidates. However, this approach requires a preprocessing model where correlated randomness is available to the parties. To compute this correlated randomness, a protocol such as [15] would need to be used, which would add two or more rounds and make the overall protocol not round-optimal. The question of upgrading security to full malicious security remains an open problem in [25].

Verifiable OPRFs. The verifiability notion for OPRFs, where the client is guaranteed that the evaluation is consistent with the server's committed key, originates from the UC functionality of Jarecki et al. [36] and is standardized in RFC 9497 [23]; Casacuberta et al. [17] systematize the landscape of OPRF security notions. Tyagi et al. [56] realize verifiability by an embedded NIZK proof in the classical setting. In the post-quantum setting, Eskin et al. [26] formalize verifiability via a key-binding property of the committed key, and Yang et al. [61] extend their batched OPRF to a verifiable one; both are designated-verifier. Our VOPRF follows the committed-key semantics while providing public verifiability: anyone holding the key commitment can verify the batched proof.

Proving correct garbling. Using zero-knowledge proofs to authenticate garbled circuits was put forward by Abascal et al. [1], who built non-interactive actively secure 2PC by proving correct garbling with the hash-based Ligero proof system. Chawdhuri and Prabhakaran [19] design proof-friendly garbled circuits based on elliptic-curve groups to reduce the cost of SNARK proofs of correct garbling; their construction is not post-quantum secure and their proof additionally covers the OT messages, which we show to be unnecessary in the OPRF setting. In contrast, we prove correct garbling of the unmodified AES circuit with a STARK, keeping post-quantum security, public verifiability, and offline precomputability.

II. PRELIMINARIES

In this work, we use λ to denote the security parameter. All algorithms will be considered as probabilistic algorithms unless explicitly. We let $\text{negl}(\lambda)$ denote a negligible function, i.e. a function that is $\lambda^{-\omega(1)}$. Further, we let $\text{poly}(\lambda)$ denote a polynomial function, i.e. a function that is $\mathcal{O}(\lambda^c)$. A PPT algorithm is a probabilistic algorithm with polynomial running time in the security parameter λ . Let S be an infinite set, and let $X = \{X_s\}_{s \in S}$ and $Y = \{Y_s\}_{s \in S}$ be distribution ensembles. We say that X and Y are computationally indistinguishable, denoted $X \stackrel{c}{=} Y$, if for every PPT distinguisher D and all sufficiently large $s \in S$, $|\Pr[D(X_s) = 1] - \Pr[D(Y_s) = 1]|$ is negligible in $|s|$.

For a distribution $\mathcal{D}$, we write $x \leftarrow_{\mathcal{D}}$ to denote that x is sampled according to the distribution $\mathcal{D}$. For a finite set S , we

write $x \leftarrow_{\S} S$ to indicate sampling uniformly at random from set S . In particular, if $S = \{0, 1\}^\lambda$, x is a random λ -bit string. For a probabilistic algorithm F , we denote by $a \leftarrow F(x)$ the event of obtaining a by invoking F on input x and a uniformly chosen random coin.

We use $=$ to denote equality and $:=$ to denote assignment. For $n \in \mathbb{N}$, $[n] := \{1, \dots, n\}$. We denote vectors in bold, whereas scalars are not bold.

A. Two-round, 1-out-of-2 Oblivious Transfer with Malicious Security

1-out-of-2 Oblivious Transfer [50] is a protocol between two parties a sender and a receiver. The sender inputs two secret messages m_0, m_1 , and the receiver inputs a secret choice bit $\gamma \in \{0, 1\}$. The protocol allows the receiver to get the message m_γ and guarantees that the receiver learns nothing about another message $m_{1-\gamma}$, whereas the sender learns nothing about the choice bit γ . For simplicity, we omit the public parameter of OT below. The OT with malicious security can be constructed from an indistinguishability under chosen-plaintext attack (IND-CPA)-secure key encapsulation mechanism (KEM) in random oracle model [45].

Definition 1. (OT). A two-round, 1-out-of-2 OT with malicious security consists of PPT algorithms (R_{OT}, S_{OT}, E_{OT}) with the following syntax:

1. $R_{OT}(\gamma)$: Given a choice bit $\gamma \in \{0, 1\}$, outputs a message OT_1 and a secret state st_{OT} .
2. $S_{OT}(m_0, m_1, OT_1)$: Given two strings $m_0, m_1 \in \{0, 1\}^\lambda$, and a message OT_1 , outputs a message OT_2 .
3. $E_{OT}(st_{OT}, OT_2)$: Given the secret state st_{OT} , and a message OT_2 , outputs a message m' .

We require the following properties:

Correctness: For any $\gamma \in \mathbb{F}_2$, $m_0, m_1 \in \{0, 1\}^\lambda$,

$$\Pr \left[E_{OT}(st_{OT}, OT_2) = m_\gamma \mid \begin{array}{l} (OT_1, st_{OT}) \leftarrow R_{OT}(\gamma) \\ OT_2 \leftarrow S_{OT}(m_0, m_1, OT_1) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

Receiver Privacy:

$$\begin{aligned} & \{ OT_1 \mid (OT_1, st_{OT}) \leftarrow R_{OT}(0) \} \\ & \stackrel{c}{=} \{ OT_1 \mid (OT_1, st_{OT}) \leftarrow R_{OT}(1) \}. \end{aligned}$$

Sender Privacy: For any ppt adversary R^* , there exists a ppt algorithm SimOT , such that:

$$(S, R^*)_{\Pi} \stackrel{c}{=} (\mathcal{F}_{OT}, \text{SimOT}),$$

where $\mathcal{F}_{OT}$ denotes the ideal functionality of OT, defined as $\mathcal{F}_{OT} : ((m_0, m_1), \gamma) \rightarrow (\perp, m_\gamma)$.

Correctness ensures that the output of the receiver corresponds exactly to the message chosen by the receiver if both parties follow the protocol honestly. *Receiver Privacy* guarantees that a malicious server cannot distinguish between requests generated by an honest receiver using different choice bits γ . Intuitively, *Sender Privacy* guarantees that an honest response message OT_2 to a malicious receiver should not reveal any information about $m_{1-\gamma}$. More specifically, there exists a simulator SimOT that interacts externally with an ideal functionality $\mathcal{F}_{OT}$ (which returns one of the correct messages chosen by SimOT) and internally with the adversary R^* . The

simulator should be capable of simulating the output of R^* during the real execution with an honest sender.

B. Garbled Circuits

Garbled Circuit, first presented by Andrew Yao [62], is a well-known generic technique for secure multi-party computation. It provides a compiler that takes any circuit C and constructs a protocol for securely computing the circuit C between a garbler and an evaluator in the presence of semi-honest adversaries. We adapt the definition of garbled circuits in [22] to our setting where both parties securely compute the AES encryption function.

In our setting, the garbler holds a secret input k , while the evaluator holds a secret input x and wishes to evaluate $C(k, x)$ obliviously. The garbler begins by garbling a public circuit C to create a garbled circuit GC and generating random labels for the input wires. If the evaluator obtains GC and the input labels corresponding to k and x , the evaluator can evaluate the correct output $C(k, x)$. The *privacy* property requires that GC and the input labels reveal no more information about k than the output $C(k, x)$. It is important to note that, in our case, the circuit C is made public to apply half-gate garbling optimization as described by Zahur et al. [63] and Guo et al. [31], leading to higher efficiency while C is kept secret from the evaluator in the classical construction. For a more detailed explanation of Garbled Circuits and related terminology, readers can refer to [22], [63].

Definition 2. (GC). Let C be a deterministic circuit that receives two inputs $k \in \{0, 1\}^\lambda$, $x \in \{0, 1\}^\lambda$. Then a garbled circuit for C consists of PPT algorithms $(\text{GenGC}, \text{EvalGC})$ with the following syntax:

1. $\text{GenGC}(1^\lambda, C)$: Given the circuit C , outputs a garbled circuit GC , 2λ input-wire labels of k , $\delta_{1,0}, \delta_{1,1}, \dots, \delta_{\lambda,0}, \delta_{\lambda,1} \in \{0, 1\}^\lambda$, and 2λ input-wire labels of x , $\omega_{1,0}, \omega_{1,1}, \dots, \omega_{\lambda,0}, \omega_{\lambda,1} \in \{0, 1\}^\lambda$.
2. $\text{EvalGC}(GC, \{\delta_i\}_{i=1}^\lambda, \{\omega_i\}_{i=1}^\lambda)$: Given a garbled circuit GC , and 2λ input labels, outputs a evaluation value $z \in \{0, 1\}^\lambda$.

We require the following properties:

Correctness: For any circuit C that satisfies the above condition, any input $k = k_1 \dots k_\lambda \in \{0, 1\}^\lambda$, $x = x_1 \dots x_\lambda \in \{0, 1\}^\lambda$, and any $(GC, \{\delta_{i,0}, \delta_{i,1}\}_{i=1}^\lambda, \{\omega_{i,0}, \omega_{i,1}\}_{i=1}^\lambda)$ output by $\text{GenGC}(1^\lambda, C)$,

$$\Pr [\text{EvalGC}(GC, \{\delta_{i,k_i}\}_{i=1}^\lambda, \{\omega_{i,x_i}\}_{i=1}^\lambda) = C(k, x)] \geq 1 - \text{negl}(\lambda).$$

Privacy: For any circuit C that satisfies the above condition, any input $k = k_1 \dots k_\lambda \in \{0, 1\}^\lambda$, $x = x_1 \dots x_\lambda \in \{0, 1\}^\lambda$, and any $(GC, \{\delta_{i,0}, \delta_{i,1}\}_{i=1}^\lambda, \{\omega_{i,0}, \omega_{i,1}\}_{i=1}^\lambda)$ output by $\text{GenGC}(1^\lambda, C)$,

$$(GC, \{\delta_{i,k_i}\}_{i=1}^\lambda, \{\omega_{i,x_i}\}_{i=1}^\lambda) \stackrel{c}{=} \text{SimGC}(1^\lambda, C, x, C(k, x)).$$

C. Pseudorandom Functions

During the 1980s, Goldreich et al. [30] made a groundbreaking discovery by demonstrating the feasibility of constructing deterministic functions that generate output resembling random choices. These functions, commonly referred to as pseudorandom functions (PRFs), have since become a fundamental cryptographic primitive, finding extensive use

across various applications and implementations. Here we adopt the definition of PRFs as provided in [5].

Definition 3. (PRFs). Let X and Y be finite sets, and let $\mathcal{F} = \{F_i : X \rightarrow Y\}$ be a function family, endowed with an efficiently sampleable distribution (more precisely, $\mathcal{F}$, X and Y are all indexed by the security parameter λ). We say that $\mathcal{F}$ is a pseudorandom function family if the following two games are computationally indistinguishable:

1. Choose a function $F \leftarrow_{\$} \mathcal{F}$ and give the adversary adaptive oracle access to $F(\cdot)$.
2. Choose a uniformly random function $U : X \rightarrow Y$ and give the adversary adaptive oracle access to $U(\cdot)$.⁴

III. OBLIVIOUS PSEUDORANDOM FUNCTION

Oblivious Pseudorandom Functions (OPRFs) are commonly defined based on simulation-based security that adheres to the real world - ideal world paradigm. To define the simulation-based security of OPRFs, it is necessary to specify the characteristics of an "ideal" OPRF, i.e., how a fully trusted party $\mathcal{F}_{\text{OPRF}}$ would perform the task while interacting with a client who holds the secret input x and a server who holds the secret key k . In the context of secure multi-party computation (MPC), an OPRF protocol for the pseudo-random function (PRF) $f_k(\cdot)$ can be seen as a secure computation of the functionality $\mathcal{F}_{\text{OPRF}} : (k, x) \rightarrow (\perp, f_k(x))$.

However, security models based on either semi-honest or malicious adversaries in the gold-standard simulation [16] do not accurately capture the security requirements of some password-based schemes, including PPSS, password hardening, and E2SE. In these cases, while the malicious server(s) are prevented from learning any information about the input x to satisfy the *client privacy* property, the adversary can still influence the final output, which violates the *verifiability* property. In contrast, the malicious model in the gold-standard simulation implies both *client privacy* and *verifiability* properties simultaneously.

Due to the heavy reliance on non-interactive zero-knowledge (NIZK) techniques, the cost required to achieve the *verifiability* property in certain protocols can be prohibitively expensive and can dominate the overall protocol cost. To address this issue, Jarecki et al. [37] originally proposed the UC definition without the *verifiability* property. In line with this approach, we degrade it and propose a game-based definition that forgoes the *verifiability* property as well. Instead, we define an *obliviousness* property that is sometimes required in password-based schemes such as password hardening and E2SE. The *obliviousness* property ensures that even if an honest client outputs an incorrect result, the malicious server cannot predict the result. It is worth noting that the *obliviousness* property implies the *client privacy* property, as any information about the input could aid the adversary in predicting the result. In fact, the extra property can be often efficiently achieved in the random oracle model (ROM).

⁴To efficiently simulate access to a uniformly random function $U : X \rightarrow Y$, one may think of a process in which the adversary's queries are "lazily" answered with independently and randomly chosen elements in Y , while keeping track of the answers so that queries made more than once are answered consistently.

In contrast to the UC definition, our protocol enables prompt extraction of the input by leveraging the rewind technique implemented in the OT module. This eliminates the need for a ticketing mechanism or non-standard assumptions to delay the extraction. However, since the UC model disallows rewinding, upgrading our protocol to a UC model without additional assumptions poses a significant challenge.

Compared to the gold-standard definition, our protocol also dispenses with the explicit definition of a PRF family, which only guarantees that the output equals the function result. This definition falls short of directly meeting the requirements of password-based schemes. In contrast, we focus on defining the *pseudorandomness* property directly, which guarantees that even if a malicious client makes up to B queries to an honest server, the adversary cannot distinguish between a correct evaluation of a secret input and a random string.

Remark 1 (Comparison with the security model of Tyagi et al. [56]). Both our game-based security model and the one proposed by Tyagi et al. [56] for their pairing-free POPRF avoid the ideal-functionality formulations inherited from UC-style definitions, but they differ in the following aspects.

- *Scope of pseudorandomness.* Their pseudorandomness definition covers arbitrary inputs and enforces a one-more-style evaluation accounting per public tag, whereas our $(\epsilon, \mathcal{D}, B)$ -*pseudorandomness* considers a secret input sampled from a distribution $\mathcal{D}$ with minimum entropy d and bounds the adversary's advantage by roughly $B \cdot 2^{-d}$. Our notion is an application-driven weakening tailored to password-based schemes such as PPSS and E2SE, where inputs inherently have bounded entropy and online guessing is the best possible attack in any case.
- *Verifiability versus obliviousness.* Their model provides *verifiability*: enforced by an NIZK proof, a malicious server cannot make an honest client accept an incorrect output. In contrast, our model forgoes verifiability and only guarantees *obliviousness*, i.e., the output of an honest client remains unpredictable to the malicious server even when it is incorrect. As discussed above, this weaker guarantee already suffices for applications such as PPSS and E2SE, where an incorrect evaluation only leads to authentication failure without leaking the secret.
- *Assumptions and post-quantum security.* Their pseudorandomness reduction relies on the newly introduced interactive gap assumption OM-Gap-SDHI, whose hardness is established only in the algebraic group model, and their construction is based on discrete-logarithm-style assumptions that do not withstand quantum attacks. In contrast, our construction is built from standard primitives (an IND-CPA-secure KEM, the AES family as a PRF, and hash functions modeled as random oracles) and admits post-quantum instantiations, e.g., from lattice-based OT.

In summary, the model of [56] provides stronger security guarantees, whereas our model trades these guarantees for post-quantum instantiability and more conservative assumptions, which is sufficient for our target applications.

Game $\text{PROPRF}_{\mathcal{D},B}^A$	Oracle $\text{LimitS}(\text{req})$
$b \leftarrow_{\$} \{0,1\}$	$i \leftarrow i + 1$
$i \leftarrow 0$	if $i \leq B$ then
$\text{pp} \leftarrow \text{Setup}(1^\lambda)$	return $\text{SEval}(\text{pp}, \text{req}, k)$
$k \leftarrow \text{KeyGen}(\text{pp})$	return $\perp$
$x^* \leftarrow_{\$} \mathcal{D}$	
$(\text{st}, \text{req}) \leftarrow \text{CEval}_1(\text{pp}, x^*)$	
$\text{rep} \leftarrow \text{SEval}(\text{pp}, \text{req}, k)$	
$y_0 \leftarrow \text{CEval}_2(\text{pp}, \text{rep}, \text{st})$	
$y_1 \leftarrow_{\$} \{0,1\}^\lambda$	
$b' \leftarrow \mathcal{A}^{\text{LimitS}(\cdot)}(y_b)$	
return $(b = b')$	

Fig. 1. The security game $\text{PROPRF}_{\mathcal{D},B}^A$ for the pseudorandomness property of OPRF.

We make some syntactic modifications to the game-based definition of E2S2 and introduce a formal definition for OPRF below.

Definition 4. (OPRF). A (two-round) oblivious pseudorandom function (OPRF) consists of PPT algorithms (Setup , KeyGen , CEval_1 , SEval , CEval_2 , FEval) with the following syntax:

1. $\text{Setup}(1^\lambda)$: Given the security parameter λ , outputs public parameter pp .
2. $\text{KeyGen}(\text{pp})$: Given public parameter pp , outputs server's secret key $k \in \{0,1\}^\lambda$.
3. $\text{CEval}_1(\text{pp}, x)$: Given public parameter pp , and client's secret input $x \in \{0,1\}^\eta$, outputs a request message req and a secret state st .
4. $\text{SEval}(\text{pp}, \text{req}, k)$: Given public parameter pp , a request message req , and the secret key k , outputs a response message rep .
5. $\text{CEval}_2(\text{pp}, \text{rep}, \text{st})$: Given public parameter pp , a response message rep and the secret state st , outputs a PRF evaluation y .
6. $\text{FEval}(k, x)$: Given secret key k , and input x , outputs a PRF evaluation $\mathcal{F}_k(x)$.

We require the following properties:

Definition 5. (Correctness (OPRF)). We call an OPRF with correctness if for any $x \in \{0,1\}^\eta$,

$$\Pr \left[\text{FEval}(k, x) = y \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ k \leftarrow \text{KeyGen}(\text{pp}) \\ (\text{st}, \text{req}) \leftarrow \text{CEval}_1(\text{pp}, x) \\ \text{rep} \leftarrow \text{SEval}(\text{pp}, \text{req}, k) \\ y \leftarrow \text{CEval}_2(\text{pp}, \text{rep}, \text{st}) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

Definition 6. (Pseudorandomness (OPRF)). let $\mathcal{D}$ be a distribution of input x and B be the maximum number of the adversary to query the server. We call an OPRF with $(\epsilon, \mathcal{D}, B)$ -pseudorandomness if the probability of any PPT adversary to win the following **Game PROPRF** is ϵ .

Definition 7. (Obliviousness (OPRF)). let $\mathcal{D}$ be a distribution of input x and g be the maximum number of the adversary to guess output. We call an OPRF with $(\epsilon, \mathcal{D}, g)$ -obliviousness if the probability of any PPT adversary to win the following **Game OBOPRF** is ϵ .

Game $\text{OBOPRF}_{\mathcal{D},g}^A$
$b \leftarrow_{\$} \{0,1\}$
$\text{pp} \leftarrow \text{Setup}(1^\lambda)$
$k \leftarrow \text{KeyGen}(\text{pp})$
$x^* \leftarrow_{\$} \mathcal{D}$
$(\text{st}, \text{req}) \leftarrow \text{CEval}_1(\text{pp}, x^*)$
$(\text{st}', \text{rep}) \leftarrow \mathcal{A}(\text{pp}, \text{req}, k)$
$y_0 \leftarrow \text{CEval}_2(\text{pp}, \text{rep}, \text{st})$
$(y_1, \dots, y_g) \leftarrow \mathcal{A}(\text{pp}, \text{req}, k, \text{st}', \text{rep})$
for $i = 1, \dots, g$
if $y_i = y_0$ then
return 1
return 0

Fig. 2. The security game $\text{OBOPRF}_{\mathcal{D},g}^A$ for the obliviousness property of OPRF.

A. Verifiable Oblivious Pseudorandom Function

The OPRF definition above forgoes verifiability: a malicious server may cause an honest client to output an incorrect evaluation. We now extend it to a *verifiable* OPRF (VOPRF), in which the server commits to its key once and for all, and every accepted evaluation is guaranteed to be consistent with the committed key. Our formulation follows the committed-key semantics of [23], [36], is cast in the game-based style of [56], and adopts the modular decomposition of [26], in which verifiability follows from the correctness of each evaluation together with a key-binding property of the commitment. Differently from previous game-based notions, our verifiability game (i) quantifies over worst-case inputs chosen by the adversary rather than inputs sampled from a distribution, (ii) lets the adversary observe whether the client accepts or rejects, and (iii) covers batches of evaluations under a single committed key.

Remark 2 (On aborts). The obliviousness game above provides the adversary with no feedback about the client's computation, and accordingly our GC-OPRF protocol is designed so that the client *never aborts*: CEval_2 always outputs $y = H_2(x, \cdot)$ whatever the garbled evaluation returns. This is essential, since an observable abort could depend on the client's input (a selective-failure attack), which the one-shot game does not model. In contrast, a VOPRF client does reject malformed responses, and we therefore expose the accept/reject bit to the adversary in the verifiability game below and randomize the client's input encoding so that aborts are statistically independent of the input (see Section V).

Definition 8. (Extractable PRF Key Commitment). An extractable key commitment scheme for a PRF family $\mathcal{F} = \{F_k\}_{k \in \{0,1\}^\lambda}$ consists of PPT algorithms (KCCom , KCVerify) with the following syntax:

1. $\text{KCCom}(k)$: Given a key k , outputs a commitment cm and a proof π_k .
2. $\text{KCVerify}(\text{cm}, \pi_k)$: Given a commitment and a proof, outputs 1 (accept) or 0 (reject).

We require the following properties:

Correctness: For any k and $(\text{cm}, \pi_k) \leftarrow \text{KCCom}(k)$, it holds that $\text{KCVerify}(\text{cm}, \pi_k) = 1$ with overwhelming probability.

Key binding: There exist a PPT extractor $\mathcal{E}$ and an efficiently computable predicate Wellformed such that for any PPT ad-

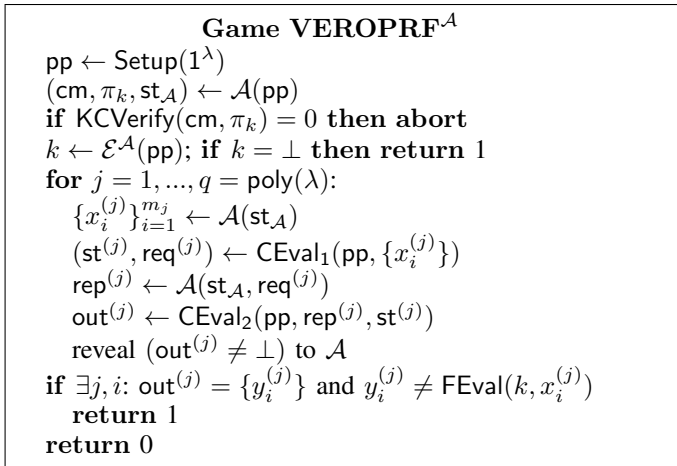


Fig. 3. The security game VEROPRF^A for the verifiability property of batch VOPRF.

versary $\mathcal{A}$,

$$\Pr \left[\begin{array}{c|c} \text{KCVerify}(\text{cm}, \pi_k) = 1 \\ \wedge \neg \text{Wellformed}(\text{cm}, k) \end{array} \middle| \begin{array}{c} (\text{cm}, \pi_k) \leftarrow \mathcal{A}(1^\lambda) \\ k \leftarrow \mathcal{E}^{\mathcal{A}}(1^\lambda) \end{array} \right] \leq \text{negl}(\lambda)$$

In our instantiation, cm consists of ciphertexts $\{F_k(i)\}$ on public points together with a hash commitment of k , the predicate $\text{Wellformed}(\text{cm}, k)$ checks the consistency of cm with k , and key binding reduces to the knowledge soundness of the proof system together with the uniqueness of keys consistent with the public ciphertexts.

Definition 9. (Batch Verifiable OPRF). A batch verifiable OPRF consists of PPT algorithms Setup, KeyGen, KeyCom, CEval₁, SEval, CEval₂, PubVerify, and FEval with the following syntax:

1. Setup(1^λ), KeyGen(pp), FEval(k, x): as in the OPRF definition.
2. KeyCom(k): One-time and reusable. Given the secret key k , outputs a key commitment cm and a proof π_k .
3. CEval₁(pp, $\{x_i\}_{i=1}^m$): Given a batch of m inputs, outputs a request message req and a secret state st .
4. SEval(pp, req, k): Given the request and the secret key, outputs a response rep containing the evaluation material and a batch proof π .
5. CEval₂(pp, rep, st): Outputs a batch $\{y_i\}_{i=1}^m$ of evaluations, or $\perp$ (reject).
6. PubVerify(cm, π): Anyone holding cm verifies the batch proof, i.e., public verifiability.

We require correctness as in the OPRF definition, pseudorandomness and obliviousness with respect to the final outputs $y_i = H_2(x_i, \text{AES}(k, H_1(x_i)))$, and the verifiability property defined below.

Definition 10. (Verifiability). A batch VOPRF has *verifiability* if the probability of any PPT adversary $\mathcal{A}$ to win the game VEROPRF in Figure 3 is $\text{negl}(\lambda)$, where the key k is extracted from the adversary's (accepted) key commitment, and the adversary chooses worst-case inputs adaptively and observes the accept/reject bit of every session.

The following lemma states that verifiability follows from the two layers of our design; its structure follows the decomposition of [26], and its proof for our construction is given in

1. Setup(1^λ): Setup the OT protocol if needed. Choose two hash functions $H_1 : \{0, 1\}^\eta \rightarrow \{0, 1\}^\lambda$, $H_2 : \{0, 1\}^\eta \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$, which are both considered as random oracles. Output the public parameter $\text{pp} := (\lambda, n, H_1, H_2, (\text{OTpp}))$. We omit OTpp in the following for simplicity.
2. KeyGen(pp): Server picks a secret key $k \leftarrow_{\$} \{0, 1\}^\lambda$ and outputs k .
3. CEval₁(pp, x): Client who holds input $x \in \{0, 1\}^\eta$, computes $x' := H_1(x) = x'_1 \cdots x'_\lambda$, and gets $(\text{OT}_{1,i}, \text{st}_{\text{OT},i}) \leftarrow \text{R}_{\text{OT}}(x'_i)$, where $x'_i \in \{0, 1\}$, $i = 1, \dots, \lambda$. Then the client outputs $\text{req} := \{\text{OT}_{1,i}\}_{i=1}^\lambda$ and a secret state $\text{st} := (x, \{\text{st}_{\text{OT},i}\}_{i=1}^\lambda)$. *// Client requests OT for the GC label corresponding to input x .*
4. SEval(pp, req, k): Server gets $(\text{GC}, \{\delta_{i,0}, \delta_{i,1}\}_{i=1}^\lambda, \{\omega_{i,0}, \omega_{i,1}\}_{i=1}^\lambda) \leftarrow \text{GenGC}(1^\lambda, \text{AES}(\cdot, \cdot))$ and $\text{OT}_{2,i} \leftarrow \text{S}_{\text{OT}}(\omega_{i,0}, \omega_{i,1}, \text{OT}_{1,i})$. Then the server outputs $\text{rep} := (\text{GC}, \{\delta_{i,k_i}\}_{i=1}^\lambda, \{\text{OT}_{2,i}\}_{i=1}^\lambda)$. *// Server responses the OT request and gives the garbled circuits to client.*
5. CEval₂(pp, rep, st): Client gets $m'_i \leftarrow \text{E}_{\text{OT}}(\text{st}_{\text{OT},i}, \text{OT}_{2,i})$ first and then outputs $y := H_2(x, \text{EvalGC}(\text{GC}, \{\delta_{i,k_i}\}_{i=1}^\lambda, \{m'_i\}_{i=1}^\lambda))$. *// Client gets corresponding label from OT and finally gets the value of OPRF from garbled circuits.*
6. FEval(k, x): Outputs $H_2(x, \text{AES}(k, H_1(x)))$.

Fig. 4. The construction of GC-OPRF.

Section V.

Lemma 1 (Composition). Let a batch VOPRF construction satisfy (i) per-evaluation correctness against a malicious server, i.e., in any accepted session every output label of the garbled evaluation decodes to the value prescribed by the garbled tables, and the garbled tables compute the correct function under some key; and (ii) key binding of the key commitment, i.e., all accepted proofs are consistent with the unique extracted key k . Then the construction satisfies verifiability.

IV. CONSTRUCTION OF GC-OPRF AND SECURITY PROOF

In this section, we introduce GC-OPRF, a protocol for constructing Oblivious Pseudorandom Functions (OPRFs) from Garbled Circuits (GC) and Oblivious Transfer (OT) in the Random Oracle Model. The protocol is depicted in Figure 4. By utilizing Kyber-based OTs from lattice assumptions in our implementation, the construction relies exclusively on post-quantum computational assumptions.

Next, we turn our attention to the security of GC-OPRF. Specifically, we analyze its security in the presence of an intentionally malicious adversary. Our analysis shows that GC-OPRF achieves malicious security under the assumption that the underlying OTs are maliciously secure while the Garbled Circuit is semi-honest secure. *Correctness* of GC-OPRF in Figure 4 is straightforward from *correctness* of OT and GC. Now we show formally that the GC-OPRF enjoys *pseudorandomness* and *obliviousness*.

A. Pseudorandomness

Theorem 1. Assuming that $H_1 : \{0, 1\}^\eta \rightarrow \{0, 1\}^\lambda$, $H_2 : \{0, 1\}^\eta \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ are both random oracles, that the family $\{\text{AES}(k, \cdot)\}_{k \leftarrow_{\$} \{0, 1\}^\lambda}$ is a pseudorandom function

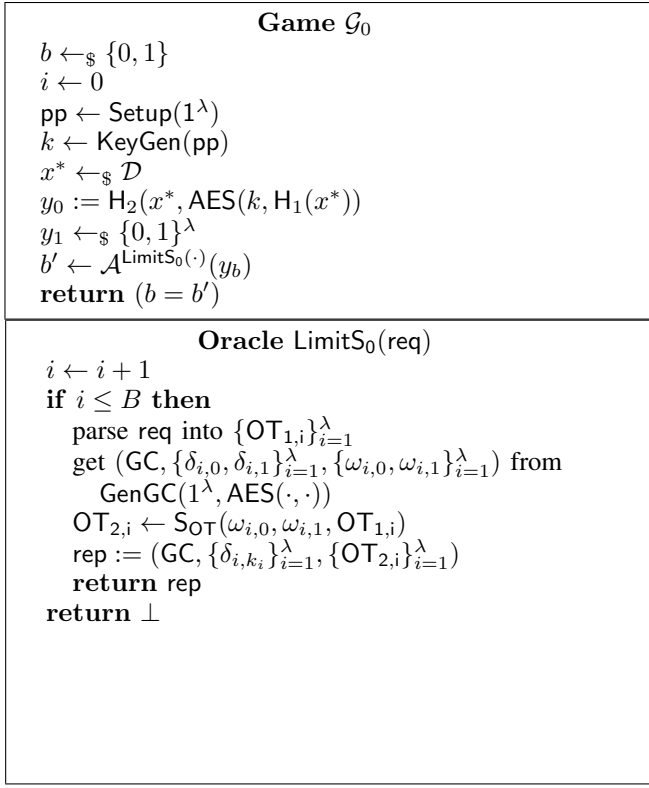


Fig. 5. The security game $\mathcal{G}_0$ in the proof of pseudorandomness, where the syntax is replaced with concrete algorithms in GC-OPRF.

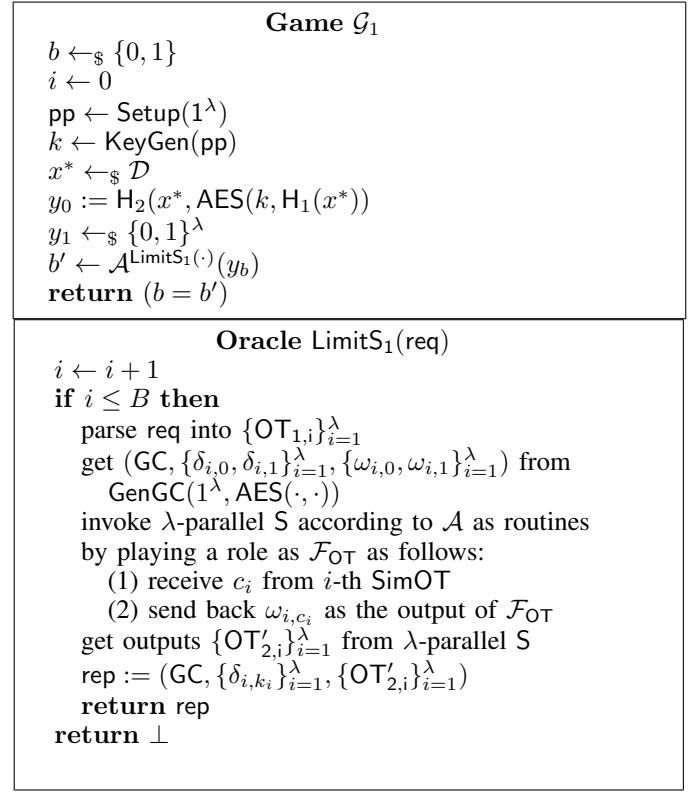


Fig. 6. The security game $\mathcal{G}_1$ in the proof of pseudorandomness, where $\{\text{OT}'_{2,i}\}_{i=1}^\lambda$ is generated by λ parallel subroutines S.

family mapping $\{0, 1\}^\lambda$ to $\{0, 1\}^\lambda$, and that there exists OT with *correctness* and *sender privacy*, GC with *correctness* and *privacy*, then for any $\mathcal{D}, B$ (that satisfies $B \cdot 2^{-d} \leq 1$), the construction from Figure 1 realizes an OPRF with $(\epsilon, \mathcal{D}, B)$ -*pseudorandomness* such that

$$\epsilon \leq 1/2 + B \cdot 2^{-d-1} + \text{negl}(\lambda),$$

where d is the minimum entropy of the distribution $\mathcal{D}$.

Proof: Define $\mathcal{G}_0, \mathcal{G}_1, \mathcal{G}_2, \mathcal{G}_3$ to be the games described in Figure 5-8, where $U(\cdot)$ in $\mathcal{G}_3$ is a uniformly chosen random function from the set of all functions mapping $\{0, 1\}^\lambda$ to $\{0, 1\}^\lambda$. The subroutine S, which is internally invoked by oracles $\text{LimitS}_1, \text{LimitS}_2$, and LimitS_3 is constructed based on a specific $\mathcal{A}$. We provide a detailed discussion on the existence of S for any specific $\mathcal{A}$ and describe the construction of S in later proofs.

$\mathcal{G}_0$ is a modification of original **Game $\text{PROPRF}_{\mathcal{D}, B}^{\mathcal{A}}$** , where the syntax is replaced with concrete algorithms in GC-OPRF. The distinguishing advantage between $\mathcal{G}_0$ and **Game $\text{PROPRF}_{\mathcal{D}, B}^{\mathcal{A}}$** is negligible because of *correctness* of OPRF.

Now we prove that the distinguishing advantage between $\mathcal{G}_0$ and $\mathcal{G}_1$ is also negligible because of *sender privacy* of OT. Note that the only difference between $\mathcal{G}_0$ and $\mathcal{G}_1$ lies in $\{\text{OT}_{2,i}\}_{i=1}^\lambda$ and $\{\text{OT}'_{2,i}\}_{i=1}^\lambda$, which are the segments of the transcripts from the corresponding OT protocols. $\{\text{OT}_{2,i}\}_{i=1}^\lambda$ is generated honestly using S_{OT} , while $\{\text{OT}'_{2,i}\}_{i=1}^\lambda$ is generated by λ parallel subroutines S.

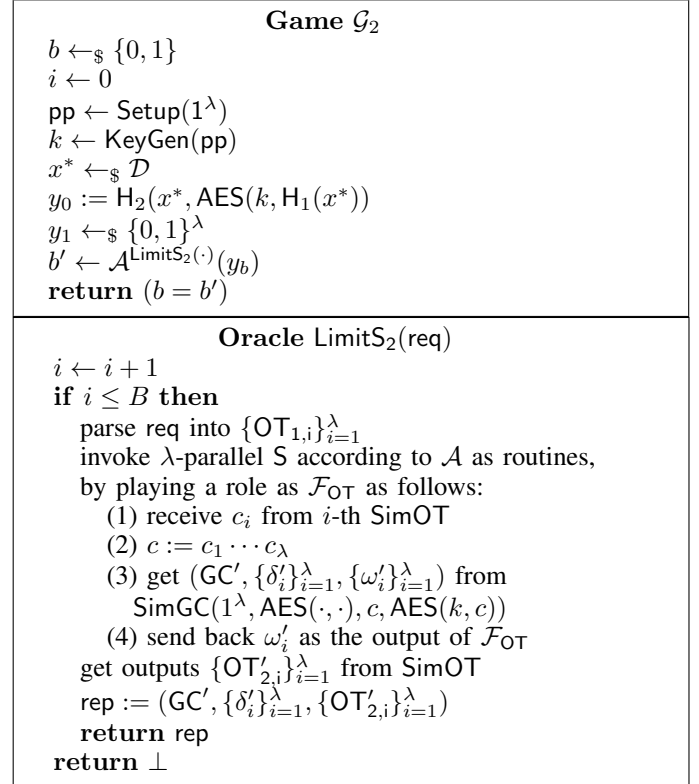


Fig. 7. The security game $\mathcal{G}_2$ in the proof of pseudorandomness, where the transcripts of garbled circuits are generated by SimGC .

Game $\mathcal{G}_3$
$b \leftarrow_{\mathcal{S}} \{0, 1\}$ $i \leftarrow 0$ $pp \leftarrow \text{Setup}(1^\lambda)$ $k \leftarrow \text{KeyGen}(pp)$ $x^* \leftarrow_{\mathcal{D}}$ $y_0 := H_2(x^*, U(H_1(x^*)))$ $y_1 \leftarrow_{\mathcal{S}} \{0, 1\}^\lambda$ $b' \leftarrow \mathcal{A}^{\text{LimitS}_3(\cdot)}(y_b)$ return $(b = b')$
Oracle $\text{LimitS}_3(\text{req})$
$i \leftarrow i + 1$ if $i \leq B$ then parse req into $\{\text{OT}_{1,i}\}_{i=1}^\lambda$ invoke λ -parallel S according to $\mathcal{A}$ as routines, by playing a role as $\mathcal{F}_{\text{OT}}$ as follows: (1) receive c_i from i -th SimOT (2) $c := c_1 \cdots c_\lambda$ (3) get $(\text{GC}', \{\delta'_i\}_{i=1}^\lambda, \{\omega'_i\}_{i=1}^\lambda)$ from $\text{SimGC}(1^\lambda, \text{AES}(\cdot, \cdot), c, U(c))$ (4) send back ω'_i as the output of $\mathcal{F}_{\text{OT}}$ get outputs $\{\text{OT}'_{2,i}\}_{i=1}^\lambda$ from SimOT $\text{rep} := (\text{GC}', \{\delta'_i\}_{i=1}^\lambda, \{\text{OT}'_{2,i}\}_{i=1}^\lambda)$ return rep return $\perp$

Fig. 8. The security game $\mathcal{G}_3$ in the proof of pseudorandomness, where the computation of AES encryption is replaced with a random function.

Considering a simple case for only a single execution of OT instead of λ parallel OTs, we provide the specific construction of the subroutine S for any adversary $\mathcal{A}$. Recall the definition of *sender privacy* of OT. For any malicious receiver R^* , there exists a simulator SimOT that interacts both externally with an ideal functionality $\mathcal{F}_{\text{OT}}$ and internally with the adversary R^* and the simulator is able to simulate the output of R^* in the real execution with a honest sender. Then, S plays a role as a malicious receiver R^* of OT, who simply forwards the OT transcript between $\mathcal{A}$ and the simulator SimOT and outputs the transcript OT_2 . By the definition, there exists a simulator SimOT that simulates R^* 's output OT_2 in the real execution. Note that if S forwards the simulated transcript OT'_2 from SimOT to $\mathcal{A}$, $\mathcal{A}$ cannot distinguish it with the transcript OT_2 in the real execution. Otherwise, we can easily invoke $\mathcal{A}$ to break *sender privacy* of OT. Therefore, S acts as a malicious receiver R^* in the OT protocol, as described earlier, with the simulator SimOT that has access to an ideal functionality $\mathcal{F}_{\text{OT}}$ supported by the oracle LimitS_1 . S finally produces the same output as SimOT .

Return to our case for λ parallel OTs, the oracle LimitS_1 invokes λ parallel S subroutines for each OT, and provide the internal simulator SimOT with an ideal functionality $\mathcal{F}_{\text{OT}}$ honestly. Then the oracle gets outputs $\{\text{OT}'_{2,i}\}_{i=1}^\lambda$ from λ parallel S and generates the response rep for $\mathcal{A}$. We claim that if there exists a distinguisher $\mathcal{D}$ that can distinguish between $\mathcal{G}_0$ and $\mathcal{G}_1$, then there is also a distinguisher $\mathcal{D}'$ that breaks the *sender privacy* with respect to OT. Intuitively, $\{\text{OT}_{2,i}\}_{i=1}^\lambda$ and $\{\text{OT}'_{2,i}\}_{i=1}^\lambda$ are computationally indistinguishable due to the independence of the λ OTs. Formally, we can define

a sequence of hybrids where $\{\text{OT}'_{2,i}\}_{i=1}^j$ is generated by S and $\{\text{OT}'_{2,i}\}_{i=j+1}^\lambda$ is generated honestly in hybrid j and the distinguisher $\mathcal{D}$ should have the ability to distinguish hybrid j from hybrid $j-1$ for a specific value of j . Then $\mathcal{D}'$ invokes $\mathcal{D}$ by simulating hybrid j and hybrid $j-1$ and it outputs the same result as $\mathcal{D}$.

Then, we claim that $\mathcal{G}_1 \stackrel{c}{=} \mathcal{G}_2$. We can observe that the modifications involve generating the output of $\mathcal{F}_{\text{OT}}$ and the output of the oracle from SimGC instead of GenGC honestly. Therefore, the computational equivalence can be proven by constructing a reduction to the *privacy* property of GC.

Next, we prove that $\mathcal{G}_2 \stackrel{c}{=} \mathcal{G}_3$ if we assume that the family $\{\text{AES}(k, \cdot)\}_{k \leftarrow_{\mathcal{S}} \{0, 1\}^\lambda}$ is a pseudorandom function family.

To complete the proof, we use the following lemma.

Lemma 2. Assuming that the family $\{\text{AES}(k, \cdot)\}_{k \leftarrow_{\mathcal{S}} \{0, 1\}^\lambda}$ is a pseudorandom function family mapping $\{0, 1\}^\lambda$ to $\{0, 1\}^\lambda$, $\mathcal{G}_2 \stackrel{c}{=} \mathcal{G}_3$.

Proof: By Definition II-C, given an oracle $\text{AES}(k, \cdot)$ and an oracle $U(\cdot)$ is computationally indistinguishable. In order to prove the lemma, we build a PRF adversary $\mathcal{B}'$ that distinguishes the two oracles in PRF game with the same advantage that $\mathcal{B}$ has in distinguishing $\mathcal{G}_2$ from $\mathcal{G}_3$. $\mathcal{B}'$ invokes $\mathcal{B}$ and acts as the challenger of $\mathcal{G}_2$ and $\mathcal{G}_3$ for $\mathcal{B}$. $\mathcal{B}'$ follows the original games honestly except that $\mathcal{B}'$ asks the oracle $\text{AES}(k, \cdot)$ (resp., $U(\cdot)$) for $\text{AES}(k, H_1(x^*))$ and $\text{AES}(k, c)$ (resp., $U(H_1(x^*))$ and $U(c)$) instead of calculating them honestly.

In the random oracle model, $\mathcal{B}'$ records all the pairs $(c, \text{AES}(k, c))$ and $(c, U(c))$ when $\mathcal{B}$ queries LimitS_2 and LimitS_3 and checks the consistency when $\mathcal{B}$ queries the random oracle H_2 .

Notice that the two modified games are all same except for the queries of $\text{AES}(k, \cdot)$ and $U(\cdot)$. Therefore, if $\mathcal{B}$ distinguishes $\mathcal{G}_2$ from $\mathcal{G}_3$, then $\mathcal{B}'$ also distinguishes the two oracles in PRF game with the same advantage. ■

We have that $\mathcal{G}_0 \stackrel{c}{=} \mathcal{G}_3$ now. We can easily see that, adversary $\mathcal{A}$ can't get any information of $H_2(x^*, U(H_1(x^*)))$ in $\mathcal{G}_3$ if $\mathcal{A}$ don't query $H_1(x^*)$. If $\mathcal{A}$ fails to guess the correct x^* from distribution $\mathcal{D}$ within B attempts with probability of success no greater than $B \cdot 2^{-d}$, $\mathcal{A}$ have to guess $H_2(x^*, U(H_1(x^*)))$ with probability of $2^{-\lambda}$, or guess b directly with probability of $1/2$. To be specific, we denote as S an event that $\mathcal{A}$ guesses the correct x^* from distribution $\mathcal{D}$ within B attempts successfully, and we use the notations E_1 and E_2 to represent $\mathcal{A}$ successfully guessing $H_2(x^*, U(H_1(x^*)))$ and b respectively, then

$$\begin{aligned}
\epsilon &\leq \Pr[S] + \Pr[E_1 | \neg S] + \Pr[E_2 | \neg S] \\
&\leq B \cdot 2^{-d} + (1 - B \cdot 2^{-d}) \cdot 2^{-\lambda} + (1 - B \cdot 2^{-d}) \cdot 1/2 \\
&= 1/2 + B \cdot 2^{-d-1} + \text{negl}(\lambda)
\end{aligned}$$

The claim completes the proof. ■

B. Obliviousness

Theorem 2. Assuming two random oracles $H_1 : \{0, 1\}^\eta \rightarrow \{0, 1\}^\lambda$, $H_2 : \{0, 1\}^\eta \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$, and OT with

receiver privacy, then for any $\mathcal{D}, g$ (that satisfies $g \cdot 2^{-d} \leq 1$), the construction from Figure 1 realizes an OPRF with $(\epsilon, \mathcal{D}, g)$ -obliviousness such that

$$\epsilon \leq g \cdot 2^{-d} + \text{negl}(\lambda),$$

where d is the minimum entropy of the distribution $\mathcal{D}$.

Proof: Define $\mathcal{G}'_0$ to be the **Game OBOPRF** and $\mathcal{G}'_1$ to be a modification of $\mathcal{G}'_0$. Firstly, the challenger of $\mathcal{G}'_1$ generates another request req' for a random x' , i.e., $(\text{st}, \text{req}') \leftarrow \text{CEval}_1(\text{pp}, x')$. Then the challenger gives the adversary $\mathcal{A}$ req' instead of original req . We have $\mathcal{G}'_0 \stackrel{c}{=} \mathcal{G}'_1$ because of receiver privacy of OT.

Note that $y_0 = H_2(x^*, \text{EvalGC}(\cdot, \cdot, \cdot))$ in $\mathcal{G}'_1$, where H_2 is a random oracle. The adversary $\mathcal{A}$ cannot predict y_0 if they do not have any information about x^* . It means that $\mathcal{A}$ must guess the correct x^* from distribution $\mathcal{D}$ within g chances with probability of success no greater than $g \cdot 2^{-d}$, or guess y_0 directly with probability of $2^{-\lambda}$. The claim completes the proof. ■

V. GC-VOPRF: VERIFIABLE OPRF FROM GARBLED CIRCUITS

In this section, we upgrade GC-OPRF to a batch verifiable OPRF, following the two-layer design sketched in the technical overview. In the *key-binding layer* (Section V-A), the server commits to its AES key once and for all. In the *circuit-authentication layer* (Section V-B), the server proves in a single STARK per batch that all garbled circuits were correctly generated from the AES circuit and the committed key. We then present a generic compiler underlying the construction (Section V-C) and its security analysis (Section V-D).

A. The Key Commitment Gadget

Let T be a small integer (by default $T = 128$) and let H be a collision-resistant hash function. The key commitment gadget instantiates the extractable PRF key commitment of Section III-A as follows.

- **KCCom**(k): Compute $c_i := \text{AES}(k, i)$ for $i = 0, \dots, T - 1$ and $\text{cm}_k := H(k)$. Set $\text{cm} := (\text{cm}_k, c_0, \dots, c_{T-1})$. Generate a STARK proof π_k of knowledge of $k \in \{0, 1\}^\lambda$ such that $\text{cm}_k = H(k)$ and $c_i = \text{AES}(k, i)$ for all i . Output (cm, π_k) .
- **KCVerify**(cm, π_k): Accept iff the STARK proof π_k verifies with respect to the public input cm .
- **Wellformed**(cm, k): Check $\text{cm}_k = H(k)$ and $c_i = \text{AES}(k, i)$ for all i .

Lemma 3 (Key binding). Assuming the knowledge soundness of the STARK and the collision resistance of H , the gadget above is an extractable PRF key commitment.

Proof sketch: The extractor runs the STARK knowledge extractor to obtain k . Knowledge soundness guarantees that an accepted proof yields k consistent with all public ciphertexts and with cm_k , except with negligible probability. It remains to note that the extracted key is unique: for any $k' \neq k$, the probability (over the choice of k) that $\text{AES}(k', i) = \text{AES}(k, i)$

for two distinct public points i is at most $2^{-\lambda}$ by a standard counting argument over the 2^λ candidate keys, so $T \geq 2$ ciphertexts already determine the key uniquely with overwhelming probability; taking $T = 128$ leaves an overwhelming margin and additionally provides public test vectors. ■

The commitment is generated once, is reusable across arbitrarily many batches and clients, and is publicly verifiable: anyone can check π_k against cm . The one-time proving cost is that of a STARK over T AES evaluations sharing one key schedule.

B. Batch Evaluation with Proof of Correct Garbling

We now describe the batch VOPRF protocol; the full description is in Figure 9. Three design features are worth highlighting.

Hoisted key schedule. All m garbled circuits in a batch garble the same *AES datapath* circuit C , which takes the eleven round keys $\text{rk}_0, \dots, \text{rk}_{10}$ as garbler's input and the client's encoded input as evaluator's input. The server computes the round keys from k locally, and the batch proof shows *once per batch* that $\text{rk} = \text{KeySchedule}(k)$ for the committed k . This removes the key schedule (about 20% of the AND gates) from every garbled circuit and makes all m garbling statements identical in structure.

The proof does not cover the OT. The batch proof commits to every input-wire label pair via a Merkle root, and the client checks each label it receives from the OT against the committed pair. A malicious server that places an inconsistent label in an OT response only causes the client to reject: by the authenticity of the garbling scheme, an evaluation on labels from the committed sets either decodes to the value prescribed by the (correct) tables or fails to decode. Hence no part of the OT protocol needs to be proven, and the proof is independent of the OT messages and of the client's input.

Selective-failure-free input encoding. Since a VOPRF client may reject, the accept/reject bit must not depend on the client's input. The client therefore encodes $x' = H_1(x)$ using an *s-probe-resilient encoding* [44]: a randomized encoding of the λ input bits into n OT inputs such that the shares at any s probed positions are uniform and independent of x' ; the circuit recovers x' by linear recombination, which is free under free-XOR garbling. Poisoning an OT position now causes rejection with probability governed by uniform shares rather than by x' , and we show in Lemma 4 that the accept/reject bit is statistically independent of the input. The simplest instantiation splits each bit into s random shares ($n = s\lambda$); code-based instantiations [44] give $n = O(\lambda)$ with $s = \Omega(\lambda)$, so the number of OTs grows only by a small constant factor over the base protocol.

Offline/online split. The offline part of SEval (garbling and proving) depends neither on the client's input nor on the OT messages, so the server can precompute garbled circuits and proofs in advance, in a fully parallelizable manner, and serve them from a stock. Since a garbled circuit is a one-time object, stock items are consumed once and never re-served; reusing a garbled circuit across two evaluations would leak the server's

1. $\text{Setup}(1^\lambda)$: As in GC-OPRF; additionally fix a STARK proof system, a collision-resistant hash H , the share length s , and the AES datapath circuit C with garbler input $\text{rk}_0, \dots, \text{rk}_{10}$ and evaluator input $x' \in \{0, 1\}^\lambda$.
2. $\text{KeyCom}(k)$ (one-time, offline): Output $(\text{cm}, \pi_k) \leftarrow \text{KCCom}(k)$ as in Section V-A.
3. $\text{CEval}_1(\text{pp}, \{x_i\}_{i=1}^m)$: For each i , compute $x'_i := H_1(x_i)$ and encode it into n OT input bits $(b_{i,1}, \dots, b_{i,n})$ using the s -probe-resilient encoding; run $(\text{OT}_1, \text{st}_{\text{OT}}) \leftarrow \text{RO}_{\text{OT}}(b_{i,j})$ for all i, j . Output $\text{req} := \{\text{OT}_1\}$ and $\text{st} := (\{x_i\}, \{\text{st}_{\text{OT}}\})$.
4. $\text{SEval}(\text{pp}, \text{req}, k)$: *Offline part (input-independent, precomputable)*: for $i = 1, \dots, m$, garble C obtaining $(\text{GC}_i, \{\delta\}, \{\omega\})$ with wire labels; commit to all input-wire label pairs of all circuits in Merkle roots $\{R_i\}$; generate a STARK proof π for the statement in Figure 10. *Online part*: answer each OT with the corresponding label pair, $\text{OT}_2 \leftarrow \text{SOT}(\omega_{i,j,\ell,0}, \omega_{i,j,\ell,1}, \text{OT}_1)$; send $\text{rep} := (\{\text{GC}_i, \delta_{i,\text{rk}}, R_i\}, \pi, \{\text{OT}_2\})$.
5. $\text{CEval}_2(\text{pp}, \text{rep}, \text{st})$: Verify π against $(\text{cm}, \{R_i\}, \{\text{GC}_i\})$; for each i, j, ℓ , obtain the label $m'_{i,j,\ell} \leftarrow \text{EOT}(\text{st}_{\text{OT}}, \text{OT}_2)$ and check it against the committed pair in R_i ; evaluate $z_i \leftarrow \text{EvalGC}(\text{GC}_i, \{\delta_{i,\text{rk}}\}, \{m'_{i,j,\ell}\})$ (the circuit XORs the shares and computes the AES datapath). If any verification or evaluation fails, output $\perp$; otherwise output $\{y_i\}$ with $y_i := H_2(x_i, z_i)$.
6. $\text{PubVerify}(\text{cm}, \pi)$: Verify the STARK proof π against cm and the public parts of rep .
7. $\text{FEval}(k, x)$: Outputs $H_2(x, \text{AES}(k, H_1(x)))$.

Fig. 9. The construction of GC-VOPRF (batch verifiable OPRF).

Statement of the batch proof π (STARK).

Public input: key commitment cm ; Merkle roots $\{R_i\}_{i=1}^m$ of input-wire label pairs; hashes of the garbled tables $\{\text{GC}_i\}$; key-wire labels $\{\delta_{i,\text{rk}}\}$; output decoding information.

Witness: the key k ; all wire labels and garbling randomness of the m circuits.

Constraints: (i) cm opens to k and $\text{rk} = \text{KeySchedule}(k)$ (once per batch); (ii) for each circuit i and each AND gate, the half-gate ciphertexts in GC_i are computed correctly from the wire labels via the fixed-key AES gate hash; (iii) the key-wire labels of every circuit encode the same rk ; (iv) the Merkle roots $\{R_i\}$ commit to the input-wire label pairs used in (ii).

Fig. 10. The NP statement proven by the batch STARK.

key, which the server is incentivized to protect. Stock can be prepared in blocks of fixed sizes (e.g., powers of two) and combined to match the requested batch size, and is invalidated upon key rotation. The online phase remains two-round: the client sends the OT messages, and the server replies with the OT responses, the precomputed circuits, and the proof.

C. The Generic Compiler

The construction above is an instantiation of a generic compiler, which we state for completeness.

Theorem 3 (Compiler). Let OPRF be a two-round OPRF with pseudorandomness and obliviousness built from malicious-secure OT and semi-honest garbled circuits as in Section IV, let $(\text{KCCom}, \text{KCVerify})$ be an extractable PRF key commitment, and let Π be a non-interactive zero-knowledge proof of knowledge for the correct-garbling relation (Figure 10). Then the protocol of Figure 9 is a batch VOPRF satisfying correctness, pseudorandomness, obliviousness, and verifiability.

The compiler is agnostic to the proof system: a STARK yields post-quantum security and public verifiability, while Ligero-style [1] or succinct SNARK instantiations [19] yield different trade-offs.

D. Security Analysis

Lemma 4 (Selective-failure freedom). Let the client encode each input with the XOR-sharing s -probe-resilient encoding ($n = s\lambda$). For any m and any two batches $\{x_i\}, \{x_i^*\}$, the distributions of the client's accept/reject bit in the **VEROPRF** game are within statistical distance $m\lambda \cdot 2^{-(s-1)}$.

Proof sketch: Rejection can only be caused by OT positions at which the server placed an inconsistent label. For a group of positions encoding one bit $x'_{i,j}$, if the server poisons a proper subset of the s positions, rejection is triggered by uniform shares and its probability is identical under both inputs. If the server poisons all s positions of the group with the same choice bit, rejection occurs unless all shares are zero, which has probability $2^{-(s-1)}$ under $x'_{i,j} = 0$ and probability 0 under $x'_{i,j} = 1$; the difference is at most $2^{-(s-1)}$ per group. A union bound over the $m\lambda$ groups gives the claim. The same independence holds for code-based s -probe-resilient encodings with $n = O(\lambda)$ by the standard probe-resilience analysis [44], with statistical distance $m \cdot 2^{-\Omega(s)}$. ■

Theorem 4 (Verifiability). Assuming the knowledge soundness of the STARK, the authenticity of the garbling scheme, and the key binding of the key commitment (Lemma 3), the construction of Figure 9 satisfies verifiability (Definition in Figure 3) with negligible soundness error for worst-case inputs.

Proof sketch: By key binding, the challenger extracts the unique key k underlying the adversary's accepted commitment, and the winning condition of extraction failure is excluded. In any accepted session, the batch proof verifies, so by knowledge soundness all garbled tables are correct half-gate garblings of the AES datapath with key-wire labels encoding $\text{rk} = \text{KeySchedule}(k)$, and all input-wire label pairs are committed in $\{R_i\}$. Since the session is accepted, every label the client obtained from the OT matches its committed pair, so by the authenticity of the garbling scheme the evaluation decodes to $z_i = \text{AES}(k, H_1(x_i))$ and $y_i = H_2(x_i, z_i) = \text{FEval}(k, x_i)$. Hence the adversary wins with negligible probability. Finally, by Lemma 4, the accept/reject feedback observed by the adversary across sessions is statistically independent of the inputs, so the multi-session game reveals nothing beyond the single-session analysis. ■

Theorem 5 (Pseudorandomness and obliviousness). The construction of Figure 9 retains the pseudorandomness and obliviousness of GC-OPRF.

Proof sketch: The additional messages $(\text{cm}, \pi_k, \{R_i\}, \pi)$ are either public values or proofs that are zero-knowledge (resp., independent of the client's input), so the pseudorandomness analysis against a malicious client is unchanged. For obliviousness against a malicious server, the only new adversarial power with respect to the base protocol is the ability to cause observable rejects; by Lemma 4 this feedback is statistically independent of the client's input, and the rest of the proof follows the obliviousness proof of GC-OPRF via the receiver privacy of the OT. ■

VI. APPLICATIONS

OPRF is playing an increasingly important role as a fundamental building block for constructing privacy-preserving protocols and schemes. It is utilized in various applications including keyword search (KS) [29], private set intersection (PSI) [33], and password-based schemes such as PPSS [4], PAKE [39], and E2SE [20]. Here, we specifically focus on discussing password-based schemes.

Nowadays, password-based schemes have received significant attention, particularly in the client-server(s) setting. In this scenario, the client can utilize a human-memorable password for authentication to the server(s), aiming to securely accomplish specific objectives with the assistance of the server(s) without revealing the password. In the basic setting, it is often assumed that two parties share the same low-entropy password beforehand. They collaborate to generate a high-entropy cryptographic object, which provides protection against offline dictionary attacks. This defense is specifically designed to counter an active adversary who possess a dictionary (or a certain distribution) from which the password has been selected. OPRF has been established as a fundamental tool for generating additional entropy in the aforementioned setting. Despite having some inherent limitations, such as the inability to prevent online impersonation attacks, password-based schemes are still currently considered indispensable and important means of privacy protection due to their low cost and ease of implementation.

In addition to inherent limitations, password-based schemes are currently facing a new challenge: the risks posed by the advancement of quantum computers. As previously mentioned, most of these schemes heavily rely on OPRF, which is currently far from being practical in terms of providing quantum resistance. In section 4, we propose a practical post-quantum OPRF called GC-OPRF. We further demonstrate that GC-OPRF can be employed in PPSS and E2SE, leading to the first practical post-quantum implementations of PPSS and E2SE. Considering this situation, the post-quantum E2SE is straightforward because the security models of the underlying core primitive, OPRF, remain the same. We briefly discuss the definition and construction of E2SE in Appendix A.

Following this, we focus on the application of GC-OPRF in the context of PPSS. In Section VI-A, we define the syntax of PPSS, while Section VI-B provides the game-based security definition. We then present the construction of post-quantum

PPSS based on GC-OPRF in Section VI-C, followed by a formal security proof of this construction in Section VI-D.

A. Password-Protected Secret Sharing

Password-Protected Secret Sharing (PPSS) [4] was first introduced as a promising approach to safeguarding users' sensitive information in the event of adversarial compromise of the users' devices, where such devices typically store this data. The protocol enables a user to store a secret among a group of n servers, and later reconstruct the secret using a memorable password by contacting a subset of $t + 1$ servers (where $t < n$). In the event of an attack by an adversary who gains control of t or fewer servers and all communication channels, they would be unable to learn anything about the secret or the password. PPSS has a wide range of applications now, including protecting sensitive data in an untrustworthy storage environment and supporting other password-based schemes such as password management and threshold password authenticated key exchange (T-PAKE) [36].

We adopt the notation and definitions for PPSS from [37], which simplified to the case where the adversary corrupts the threshold t of servers. In our game, the password pw is sampled from a certain password distribution $\mathcal{D}$ instead of a dictionary set D .

Definition 11. PPSS scheme $(\text{ParGen}, \text{SKeyGen}, \text{INIT}, \text{REC})$ involves user $\mathcal{U}$ and n servers $\mathcal{S}_1, \dots, \mathcal{S}_n$:

ParGen : Given security parameter λ , generates public parameters crs .

SKeyGen : $\mathcal{S}_i$, given crs , generates its private state σ_i .

INIT is a protocol between $\mathcal{U}$ and $\mathcal{S}_1, \dots, \mathcal{S}_n$ s.t.

- $\mathcal{U}$ runs algorithm U_{Init} , which given crs and a password pw , outputs a private key $K \in \{0, 1\}^\lambda$.
- $\mathcal{S}_i$ runs algorithm S_{Init} , which given crs and σ_i , outputs user-specific information ω_i .

REC is a protocol between $\mathcal{U}$ and $\mathcal{S}_j$, where $j \in S$, $S \subseteq [n]$, $|S| = t + 1$ s.t.

- $\mathcal{U}$ runs algorithm U_{Rec} , which given crs and pw , outputs $K' \in \{0, 1\}^\lambda \cup \{\perp\}$.
- $\mathcal{S}_j$ runs algorithm S_{Rec} on input crs , σ_j , and ω_j .

Definition 12. (Correctness (PPSS)). We call a PPSS scheme with correctness if for any $\text{pw} \in \eta$,

$$\Pr \left[K' = K \mid \begin{array}{l} \text{crs} \leftarrow \text{ParGen}(1^\lambda) \\ \{\sigma_i\}_{i=1}^n \leftarrow \text{SKeyGen}(\text{crs}) \\ (K, \{\omega_i\}_{i=1}^n) \leftarrow \text{INIT}(\text{crs}, \text{pw}, \{\sigma_i\}_{i=1}^n) \\ K' \leftarrow \text{REC}(\text{crs}, \text{pw}, \{\sigma_j\}_{j \in S}, \{\omega_j\}_{j \in S}) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

B. Game-Based Definition of PPSS

Definition 13. (Security (PPSS)). let $\mathcal{D}$ be a distribution of password pw . A PPSS scheme with parameters (t, n) is $(\mathcal{D}, q_U, q_S, \epsilon)$ -secure if for any corrupted set $\text{CS} \subset [n]$ s.t. $|\text{CS}| = t$, the probability of any PPT $\mathcal{A}$ to win the following game is ϵ .

- (1) Set $b \leftarrow_{\mathcal{S}} \{0, 1\}$, $\text{pw} \leftarrow_{\mathcal{S}} \mathcal{D}$, $\text{crs} \leftarrow \text{ParGen}(1^\lambda)$ and $\sigma_i \leftarrow \text{SKeyGen}(\text{crs})$ for $i \in [n]$. Send crs to $\mathcal{A}$.
- (2) Let $\mathcal{U}_{\text{Init}}(\text{crs}, \text{pw})$ interact with $\mathcal{S}_{\text{Init}}(\text{crs}, \sigma_i)$ for all i , where $\mathcal{A}$ has full control over each $\mathcal{S}_i$ for $i \in \text{CS}$ (and can modify their protocols), while the channels between $\mathcal{U}$ and $\mathcal{S}_i$ for $i \notin \text{CS}$ are authenticated. Let $K^{(1)}$ be $\mathcal{U}_{\text{Init}}$'s output in the above interaction, and let $K^{(0)} \leftarrow_{\mathcal{S}} \{0, 1\}^\lambda$. Send $K^{(b)}$ to $\mathcal{A}$.
- (3) $\mathcal{A}$ outputs b' after q_U interactions with $\mathcal{U}_{\text{Rec}}^\circ(\text{crs}, \text{pw}, b, K^{(b)})$ and q_S interactions with $\mathcal{S}_{\text{Rec}}$. $\mathcal{U}_{\text{Rec}}^\circ$ runs $\mathcal{U}_{\text{Rec}}$ on input (crs, pw) but when $\mathcal{U}_{\text{Rec}}$ outputs its local output K' or $\perp$, $\mathcal{U}_{\text{Rec}}^\circ$ processes this output as follows: If $b = 1$ then it forwards $\mathcal{U}_{\text{Rec}}$'s output to $\mathcal{A}$. If $b = 0$ it forwards $K^{(0)}$ instead of K' if $\mathcal{U}_{\text{Rec}}$'s output is $K' \neq \perp$, and forwards $\perp$ otherwise.
- (4) If $b' = b$, then $\mathcal{A}$ wins the game.

C. A Post-Quantum PPSS Protocol from GC-OPRF

In this section, we describe a protocol for PPSS from GC-OPRF. $\mathcal{U}_{\text{Init}}$, $\mathcal{U}_{\text{Rec}}$, $\mathcal{S}_{\text{Init}}$ and $\mathcal{S}_{\text{Rec}}$ are not explicitly defined in the following protocol.

1. $\text{ParGen}(1^\lambda)$: Run Setup of GC-OPRF, $\text{pp} \leftarrow \text{Setup}(1^\lambda)$. Set the threshold parameters of PPSS, $t, n \in \mathbb{N}$, $t < n$. Generate an instance of commitment scheme COM. Choose a hash function $H' : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$, which is considered as a random oracle. Output the public parameter $\text{crs} := (\text{pp}, t, n, \text{COM}, H')$.
2. $\text{SKeyGen}(\text{pp})$: Each server $\mathcal{S}_i$ runs KeyGen of GC-OPRF to pick its own secret key $k_i \leftarrow \text{KeyGen}(\text{pp})$ and outputs $\sigma_i := k_i$.
3. INIT : $\mathcal{U}$ and $\mathcal{S}_1, \dots, \mathcal{S}_n$ execute the following steps:
 - (1) $\mathcal{U}$ picks $s \leftarrow_{\mathcal{S}} \{0, 1\}^\lambda$ and parses $H'(s)$ as $[r||K]$.
 - (2) $\mathcal{U}$ generates $(s_1, \dots, s_n)$ as a (t, n) Shamir's secret-sharing of s over $\{0, 1\}^\lambda$.
 - (3) $\mathcal{U}$ then runs GC-OPRF with each $\mathcal{S}_i$:
 - $\mathcal{U}$ gets $(\text{st}_i, \text{req}_i) \leftarrow \text{CEval}_1(\text{pp}, \text{pw})$ and sends req_i to corresponding $\mathcal{S}_i$;
 - Each $\mathcal{S}_i$ gets $\text{rep}_i \leftarrow \text{SEval}(\text{pp}, \text{req}_i, k_i)$ and sends rep_i to $\mathcal{U}$;
 - $\mathcal{U}$ gets $\rho_i \leftarrow \text{CEval}_2(\text{pp}, \text{rep}_i, \text{st}_i)$.
- (4) $\mathcal{U}$ computes $e_i := s_i \oplus \rho_i$ and sets $\mathbf{s} := (s_1, \dots, s_n)$, $\mathbf{e} := (e_1, \dots, e_n)$.
- (5) $\mathcal{U}$ computes $C := \text{COM}((\text{pw}, \mathbf{e}, \mathbf{s}); r)$ and sends $(\mathbf{e}, C)$ to every $\mathcal{S}_i$.
- (6) $\mathcal{U}$ outputs K . Each $\mathcal{S}_i$ sets its own user-specific information $\omega_i := (\mathbf{e}, C)$ and outputs ω_i .
4. REC : $\mathcal{U}$ and $\mathcal{S}_j$ execute the following steps, where $j \in \text{S}, \text{S} \subseteq [n], |\text{S}| = t + 1$:
 - (1) Each $\mathcal{S}_j$ sends ω_j to $\mathcal{U}$.
 - (2) $\mathcal{U}$ checks if all of ω_j are consistent. If so, $\mathcal{U}$ parses any ω_j as $(\mathbf{e}', C')$ and parses $\mathbf{e}'$ as $(e'_1, \dots, e'_n)$. Otherwise, $\mathcal{U}$ outputs $\perp$.
 - (3) $\mathcal{U}$ then runs GC-OPRF with each $\mathcal{S}_j$:
 - $\mathcal{U}$ gets $(\text{st}'_j, \text{req}'_j) \leftarrow \text{CEval}_1(\text{pp}, \text{pw}')$ and sends req'_j to corresponding $\mathcal{S}_j$;
 - Each $\mathcal{S}_j$ gets $\text{rep}'_j \leftarrow \text{SEval}(\text{pp}, \text{req}'_j, k_j)$ and sends rep'_j to $\mathcal{U}$;
 - $\mathcal{U}$ gets $\rho'_j \leftarrow \text{CEval}_2(\text{pp}, \text{rep}'_j, \text{st}'_j)$.

- (4) $\mathcal{U}$ computes $s'_j := e'_j \oplus \rho'_j$ and recovers s' and other shares s'_i , where $i \in [n] \setminus \text{S}$.
- (5) $\mathcal{U}$ sets $\mathbf{s}' := (s'_1, \dots, s'_n)$ and parses $H'(\mathbf{s}')$ as $[r'||K']$.
- (6) $\mathcal{U}$ outputs K' if $C' = \text{COM}((\text{pw}', \mathbf{e}', \mathbf{s}'); r')$ and outputs $\perp$ otherwise.

D. Security Proof

Correctness of PPSS is straightforward from *correctness* of OPRF. Now we show formally that our PPSS scheme also enjoys the *security* property.

Theorem 6. Assuming a random oracle $H' : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$, a commitment scheme COM with binding, hiding and non-malleable properties, and GC-OPRF with correctness, pseudorandomness and obliviousness, then for any $\mathcal{D}$, the construction from Figure 2 realizes a PPSS scheme with $(\mathcal{D}, q_U, q_S, \epsilon)$ -secure such that

$$\epsilon \leq 1/2 + (q_U + q_S) \cdot 2^{-d-1} + \text{negl}(\lambda),$$

where d is the minimum entropy of the distribution $\mathcal{D}$.

Proof: We first define $\mathcal{G}_0$ to be the security experiment in PPSS definition. We omit ParGen , SKeyGen and $\mathcal{A}$'s input and state here.

Game $\mathcal{G}_0$

Initialization

1. Pick $b \leftarrow_{\mathcal{S}} \{0, 1\}$, $\text{pw} \leftarrow_{\mathcal{S}} \mathcal{D}$.
2. Pick $k_i \leftarrow_{\mathcal{S}} \{0, 1\}^\lambda$ and send k_c to $\mathcal{A}$, where $c \in \text{CS}$.
3. Pick $s \leftarrow_{\mathcal{S}} \{0, 1\}^\lambda$.
4. Parse $H'(s)$ as $[r||K^{(1)}]$. Set $K^{(0)} \leftarrow_{\mathcal{S}} \{0, 1\}^\lambda$.
5. Generate $(s_1, \dots, s_n)$ as a (t, n) Shamir's secret-sharing of s over $\{0, 1\}^\lambda$.
6. For each $g \notin \text{CS}$, compute $\rho_g \leftarrow \text{FEval}(\text{pw}, k_g)$.
7. For each $c \in \text{CS}$, repeat CEval_1 t times, $(\text{st}_c, \text{req}_c) \leftarrow \text{CEval}_1(\text{pp}, \text{pw})$ and send $\{\text{req}_c\}_{c \in \text{CS}}$ to $\mathcal{A}$.
8. $\mathcal{A}$ inputs $\{\text{rep}_c\}_{c \in \text{CS}}$.
9. For each $c \in \text{CS}$, $\rho_c \leftarrow \text{CEval}_2(\text{pp}, \text{rep}_c, \text{st}_c)$.
10. Compute $e_i := s_i \oplus \rho_i$ and set $\mathbf{e} := (e_1, \dots, e_n)$, $\mathbf{s} := (s_1, \dots, s_n)$.
11. Compute $C := \text{COM}((\text{pw}, \mathbf{e}, \mathbf{s}); r)$ and send $\omega := (\mathbf{e}, C)$ and $K^{(b)}$ to $\mathcal{A}$.

Reconstruction ($\mathcal{U}_{\text{Rec}}^\circ(\text{crs}, \text{pw}, b, K^{(b)})$, repeated q_U times)

1. $\mathcal{A}$ inputs $\omega' := (\mathbf{e}', C')$ and P , where $|\text{P}| = t + 1$.
2. For each $p \in \text{P}$, repeat CEval_1 $t + 1$ times, $(\text{st}'_p, \text{req}'_p) \leftarrow \text{CEval}_1(\text{pp}, \text{pw})$ and send $\{\text{req}'_p\}_{p \in \text{P}}$ to $\mathcal{A}$.
3. $\mathcal{A}$ inputs $\{\text{rep}'_p\}_{p \in \text{P}}$.
4. For each $p \in \text{P}$, $\rho'_p \leftarrow \text{CEval}_2(\text{pp}, \text{rep}'_p, \text{st}'_p)$.
5. Parse $\mathbf{e}'$ as $(e'_1, \dots, e'_n)$ and compute $s'_p := e'_p \oplus \rho'_p$.
6. Recover s' and other shares s'_q , where $i \in [n] \setminus \text{P}$.
7. Set $\mathbf{s}' := (s'_1, \dots, s'_n)$ and parse $H'(\mathbf{s}')$ as $[r'||K']$. If $b = 0$, set $K' = K^{(0)}$.
8. Output K' if $C' = \text{COM}((\text{pw}, \mathbf{e}', \mathbf{s}'); r')$ and output $\perp$ otherwise.

Reconstruction ($\mathcal{S}_{\text{Rec}}$, repeated q_S times)

1. $\mathcal{A}$ inputs (req''_i, i) .
 2. Compute $\text{rep}''_i \leftarrow \text{SEval}(\text{pp}, \text{req}''_i, k_i)$ and sends rep''_i to $\mathcal{A}$.
- Answer**
1. $\mathcal{A}$ outputs b' . If $b' = b$, then $\mathcal{A}$ wins the game.

Now we define $\mathcal{G}_1$ to be a modification of $\mathcal{G}_0$, where each ρ_g is a uniform random λ -bit string. We claim that $\mathcal{A}$ cannot distinguish between $\mathcal{G}_0$ and $\mathcal{G}_1$ with non-negligible advantage if $\mathcal{A}$ don't query correct pw's corresponding response to S_{Rec} . We can define a sequence of hybrids to prove this claim. Let $[n] \setminus \text{CS}$ be $\{g_1, \dots, g_{n-t}\}$ and their corresponding OPRF evaluation are $\rho_1, \dots, \rho_{n-t}$. In $\text{hyb}_{0,i}$, the OPRF evaluation $\rho_1, \dots, \rho_i$ are replaced by uniform random λ -bit strings, while $\rho_{i+1}, \dots, \rho_{n-t}$ are left unmodified. Then, advantage between $\text{hyb}_{0,i}$ and $\text{hyb}_{0,i+1}$ is bounded by *pseudorandomness* of OPRF. It is not hard to see that $\mathcal{G}_0$ and $\mathcal{G}_1$ are exactly $\text{hyb}_{0,0}$ and $\text{hyb}_{0,n-t}$.

Then we define $\mathcal{G}_2$ to be a modification of $\mathcal{G}_1$, which outputs $K^{(b)}$ generated in **Initialization** phase instead of K' if $\omega' = \omega$ and $\rho'_p = \rho_p$ for any $p \in P$, and rejects otherwise. We argue that $\mathcal{A}$ cannot distinguish between $\mathcal{G}_1$ and $\mathcal{G}_2$ with non-negligible advantage if $\mathcal{A}$ don't query correct pw's corresponding response to S_{Rec} . Let Pass be the event that $C' = \text{COM}((\text{pw}, e', s'); r')$ in $U_{\text{Rec}}^\diamond$, and we can see that

- If $\omega' = \omega \wedge \rho'_p = \rho_p$, then both $\mathcal{G}_1$ and $\mathcal{G}_2$ output $K^{(b)}$;
- If $\neg(\omega' = \omega \wedge \rho'_p = \rho_p) \wedge \neg \text{Pass}$, then both $\mathcal{G}_1$ and $\mathcal{G}_2$ output $\perp$;
- If $\neg(\omega' = \omega \wedge \rho'_p = \rho_p) \wedge \text{Pass}$, then $\mathcal{G}_1$ outputs K' , while $\mathcal{G}_2$ outputs $\perp$.

Therefore, we only need to consider the third case, where $\neg(\omega' = \omega \wedge \rho'_p = \rho_p) \wedge \text{Pass}$ occurs. Next we break this event into three sub-events:

- $E_1: C' \neq C \wedge \text{Pass}$;
- $E_2: C' = C \wedge (e', s') \neq (e, s) \wedge \text{Pass}$;
- $E_3: C' = C \wedge (e', s') = (e, s) \wedge \rho'_p = \rho_p \wedge \text{Pass}$.

Firstly, we claim that the probability of E_1 happening is bounded by $q_U \cdot 2^{-d}$, with a negligible term if $\mathcal{A}$ don't query correct pw's corresponding response to S_{Rec} . Note that E_1 may occur at $\mathcal{A}$'s any query to $U_{\text{Rec}}^\diamond$, we only need to prove the probability of E_1 happening at $\mathcal{A}$'s first query is bounded by 2^{-d} , with a negligible term. It can be proved by constructing a direct reduction to COM 's *non-malleability* property.

Secondly, E_2 happens with negligible probability because of COM 's *binding* property. Thirdly, we argue that E_3 never occurs because of a conflict that $\rho'_p = s'_p \oplus e'_p = s_p \oplus e_p = \rho_p$ if $(e', s') = (e, s)$.

Next, we define $\mathcal{G}_3$ to be a modification of $\mathcal{G}_2$, C is set as $C = \text{COM}((0, e, \theta); r)$ instead of $\text{COM}((\text{pw}, e, s); r)$. On $\mathcal{A}$'s view, both are identical if $\mathcal{A}$ don't know pw, s because of COM 's *hiding* property.

Note that $\mathcal{G}_3$'s output $K^{(1)}$ is only dependent on s and $n-t$ shares of s are masked by uniform random ρ_i . Therefore, we have $\Pr[\mathcal{A} \text{ wins in } \mathcal{G}_3] \leq 1/2 + \text{negl}(\lambda)$. According to previous proof, $\mathcal{A}$ cannot distinguish between $\mathcal{G}_0$ and $\mathcal{G}_3$ with non-negligible advantage if $\mathcal{A}$ don't query correct pw's corresponding response to S_{Rec} . The probability of querying corresponding response is bounded by *obliviousness* of OPRF

because querying corresponding response implies getting correct value to guess in game **OBOPRF**. It can be proved by defining a sequence of hybrids in a similar way as before.

We conclude that $\Pr[\mathcal{A} \text{ wins in } \mathcal{G}_0] \leq 1/2 [1 + (q_U + q_S) \cdot 2^{-d} + \text{negl}(\lambda)]$. It completes the proof. $\blacksquare$

E. Unbalanced PSI and Password Breach Checking

A verifiable OPRF directly yields a PSI protocol in the client-server setting: the server fixes a key k and publishes the PRF values of its set elements; the client obviously evaluates the PRF on its own items and intersects locally. We focus on the *unbalanced* setting, where the client holds a few items and the server holds a large set. A prominent deployed example is password breach (compromised-credential) checking, in which a client learns whether its credentials appear in a service provider's breach database; deployed systems of this kind rely on classical verifiable OPRFs, which are not post-quantum secure.

Verifiability is essential here: a malicious server that evaluates under inconsistent or incorrect keys causes false negatives that are indistinguishable from honest "not found" answers, silently defeating the purpose of the check. Our GC-VOPRF provides exactly the required guarantee—every accepted evaluation is consistent with the single committed key—with the additional benefit of *public* verifiability: the key commitment can be audited by anyone, which is desirable for a public service.

Construction. The server picks $k \leftarrow \text{KeyGen}(\text{pp})$ and publishes the key commitment (cm, π_k) once. For its set S , the server computes and publishes $\{H_2(s, \text{AES}(k, H_1(s)))\}_{s \in S}$ (or a compact representation thereof); note that this part requires no garbled circuits, and the server can optionally prove the correctness of these values under the committed key with the same STARK machinery, since it knows the set. A client with items $x_1, \dots, x_m$ runs the batch VOPRF of Figure 9 with the server, obtains $\{y_i\}$, and outputs the indices with y_i in the published set. The online phase is two-round, and the client's batch is small (m between 1 and 100 in typical deployments), so the client's cost approaches m times the cost of the base GC-OPRF plus one STARK verification.

Security. Client privacy follows from the obliviousness of the VOPRF: the server learns nothing about the client's items, including via the accept/reject feedback (Lemma 4). Server-side privacy follows from pseudorandomness: the client learns only the PRF values on its own queries. Correctness against a malicious server follows from verifiability (Theorem 4): all accepted evaluations are consistent with the committed key, and the published set values are computed locally by the server under the same key (optionally with a STARK proof). We note that the breach-checking application additionally involves an outer protocol that maps the intersection to a user-facing answer; this part is orthogonal and identical to deployed classical systems.

VII. IMPLEMENTATION AND EVALUATIONS

In this section, we present the implementation of our proposed GC-OPRF and PPSS schemes, as described in Section

IV and Section VI respectively, and demonstrate their practicality through experimental evaluation. For the implementation of GC-OPRF, we leverage the MP-SPDZ library⁵ [40] which provides a specialized virtual machine and compiler designed for the specifics of multi-party computation, facilitating the implementation of garbled circuits. To instantiate the post-quantum OT protocols, we adapt the malicious secure 1-out-of-2 Kyber-based base OTs [45] and SoftSpokenOT [52] for malicious secure 1-out-of-2 OT extension, both of which are available in the libOTe library⁶. Additionally, we implement our post-quantum PPSS scheme using GC-OPRF and Shamir’s secret sharing supported by MP-SPDZ. All random oracles in GC-OPRF and PPSS, as well as the commitment scheme in PPSS, are implemented using SHA-256.

In our implementation, we utilize Kyber768 as the instantiation for Kyber-based base OTs. We employ 128-bit wire labels and AES with a block size and key length of 128 bits for the implementation of garbled circuits, incorporating the half-gate technique [31], [63]. We set the security parameter λ in this work to 128. To simulate the client and server(s) functionalities, we run separate processes on the same device. We perform experiments of our work using a ThinkPad X1 Carbon 2018, with Intel(R) Core(TM) i7-8550U CPU running at 1.80GHz and 8GB of RAM. The experiments are performed within a Ubuntu 20.04 docker container. The benchmarks are established by running each of them at least ten times and calculating the average runtime.

Scheme	Privacy Security	Post-Quantum	Client Time	Server Time
2HashDH-OPRF	Malicious	×	< 1ms	< 1ms
TFHE-OPRF	Semi-Honest	✓	256(36)ms	14.4s
GC-OPRF	Malicious	✓	9.3ms	8.5ms

TABLE II. EVALUATIONS OF OPRFS

Evaluations and comparisons of OPRFs. We begin by comparing the performance of our GC-OPRF in a single execution. Specifically, we compare it with the 2HashDH-OPRF protocol proposed by Jarecki et al. [36], which relies on the One More Diffie-Hellman (OMDH) assumption and is currently widely used in password-based schemes. Additionally, we compare it with the TFHE-OPRF protocol introduced by Albrecht et al. [2], which is based on several lattice assumptions and currently holds the title of being the fastest post-quantum OPRF. The timings of both 2HashDH-OPRF and GC-OPRF include the preprocessing phase, as we are specifically focusing on a single execution rather than an amortized setting. It is worth noting that the single execution of OPRF is a common scenario when employed in password-based schemes, particularly in our post-quantum PPSS and E2SE applications. We provide the client-side timing for the TFHE-OPRF in both settings. The client-side timing for the TFHE-OPRF is listed as 256 ms in the single execution setting and 36 ms in the amortized setting, excluding the heavy key generation process. The running time of TFHE-OPRF is tested on 96 Intel Xeon Gold 6252 CPU @ 2.10GHz cores and 768 GB of RAM, using only a single core for the client side and using 64 threads/cores for the server side [2]. Additionally, it should be emphasized that the TFHE-OPRF implementation does not include the heavy zero-knowledge arguments for malicious security. As a

⁵<https://github.com/data61/MP-SPDZ>

⁶<https://github.com/osu-crypto/libOTe>

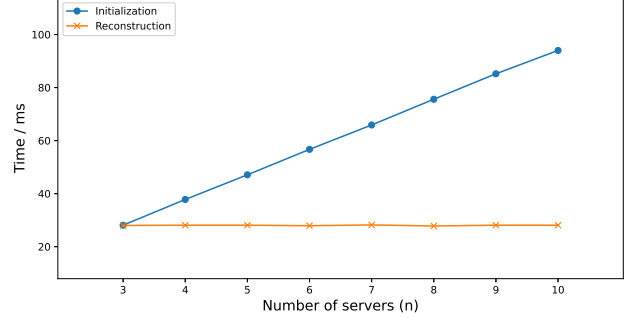


Fig. 11. Time for client in PPSS ($t = 2$)

result, the TFHE-OPRF implementation is only secure in the semi-honest model, while the 2HashDH-OPRF and GC-OPRF implementations consider a malicious security model.

As shown in Table II, our GC-OPRF protocol outperforms TFHE-OPRF in terms of both client-side and server-side computation, even in the amortized setting. In our dedicated single execution scenario, we achieve an overall speedup of more than 800 times on a weaker machine, with individual improvements of approximately 27 times for the client-side and around 1700 times for the server-side.

When it comes to the 2HashDH-OPRF, its outstanding efficiency is beyond doubt, making it a widely adopted choice in password-based schemes today. The estimated local computation cost of the 2HashDH-OPRF is even less than 1ms [20]. However, it is important to note that in a real-world scenario, the high TLS handshake and network latency, typically around 140 ms in the setting of E2SE [20], becomes a dominant factor in the overall time cost of the protocol. Taking this into consideration, the time required for our GC-OPRF protocol remains within acceptable limits, while the FHE-OPRF protocol still appears to be less feasible.

Phase	Average Client Time	Server Time
Initialization	9.4ms (divided by 3)	8.5ms
Reconstruction	9.4ms (divided by 3)	8.5ms

TABLE III. EVALUATIONS OF OUR PPSS ($n = 3, t = 2$)

Evaluations of our PPSS. Then we proceed with the implementation of our post-quantum PPSS scheme, leveraging GC-OPRF as the underlying primitive, and estimate the time cost of the scheme. To simulate the interaction between a client and n servers, we still run several separate processes on the same device. As illustrated in Section VI-C, the client engages with each server individually during both the initialization and reconstruction phases of PPSS, resulting in longer total client-side time as n and t increase. Our observations reveal that in the PPSS scheme, the OPRF component contributes to over 90% of the time overhead, exerting a dominant influence on the overall time cost, regardless of the chosen values for t and n within the range of 10. Specifically, as shown in Table III both in the initialization phase and the reconstruction of the (2, 3)-PPSS scheme, the average client-side time cost (total time divided by $n = t + 1 = 3$) is 9.4ms, while the average individual server-side time cost is 8.5ms.

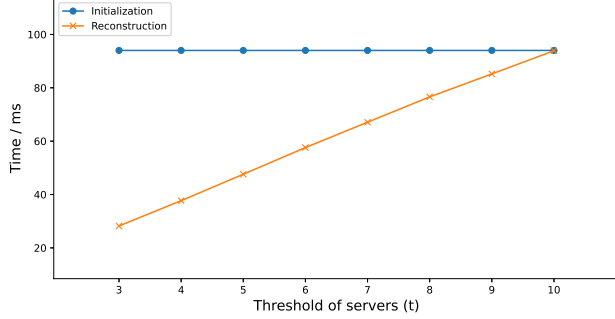


Fig. 12. Time for client in PPSS ($n = 10$)

In addition, we examine how the client-side time changes as the n and t increase in Figure 11 and Figure 12. In the case of $t = 2$, when the value of n increases, the client-side time during the initialization phase demonstrates a linear increase, while the time during the reconstruction phase remains relatively constant. On the other hand, in the scenario of $n = 10$, as the value of t increases, the client-side time during the initialization phase remains almost constant, while the time during the reconstruction phase exhibits a linear increase.

Evaluations of GC-VOPRF. We finally evaluate the verifiable extension. The VOPRF adds two components to the base protocol, which we benchmark separately: the *batch garbling* (including witness generation) and the *STARK proofs*. All benchmarks of the verifiable extension are performed on an Apple M5 Pro laptop.

For batch garbling, we measure the half-gate garbling of the AES-128 circuit with the fixed-key AES (MMO) gate hash using the ARM AES instructions. Since garbling cost is per-gate and independent of the circuit topology, we garble synthetic circuits with the gate counts of the AES-128 encryption circuit (6,553 AND gates and 24,000 XOR gates, matching the 0.2MB garbled tables of our base protocol). Garbling one circuit takes 0.21ms (amortized over a batch of 64), producing 204.8KB of garbled tables and a 1.2MB witness for the prover; evaluating one circuit takes 0.066ms on the client side.

For the STARK proofs, we implement a prototype prover and verifier using the winterfell library over the 62-bit prime field, proving N independent AES evaluations in which every state byte is constrained by an (input, S-box, xtime) triple lookup into a 256-entry table via logarithmic-derivative lookups [32], with 30 queries, blowup factor 16, grinding factor 16, and a quadratic extension field. The production target of our construction is a binary-field STARK [24], in which the AES linear layer (MixColumns, AddRoundKey) and all wiring are linear and free; the prover cost is dominated by the S-box/xtime lookups and the state wiring, which our prototype proves in full, so the measured costs faithfully model the production cost. The results are shown in Table IV: the prover scales linearly at about 0.2ms per AES evaluation, the proof size grows logarithmically, and verification takes about 3ms regardless of the batch size.

We derive the following figures for the full construction.

N (AES evals)	Prove	Proof size	Verify
16	23.0ms	43.1KB	1.2ms
256	74.6ms	67.7KB	1.6ms
1,024	194.6ms	82.9KB	3.1ms
4,096	807.5ms	96.6KB	2.5ms
16,384	5,523.8ms	114.6KB	2.8ms

TABLE IV. STARK PROTOTYPE FOR N FIXED-KEY AES EVALUATIONS (APPLE M5 PRO, WINTERFELL, F62, LOGUP).

The one-time *key commitment* ($T = 128$ AES evaluations) costs about 30ms of proving and a 50KB proof, once per key. Each garbled circuit requires 26,212 MMO gate-hash evaluations (4 per AND gate), hence about 5.2s of proving per circuit; a batch of $m = 128$ circuits requires about 11 minutes of proving on a single machine, which is entirely offline, input-independent, and embarrassingly parallel across machines or GPUs, and can be recursively aggregated into a single logarithmic-size proof. In the online phase, a client with a batch of $m = 128$ inputs evaluates m garbled circuits (8.4ms in total) and verifies one proof (about 3ms), so the amortized client cost is about 0.09ms per evaluation plus the OT cost (0.044ms per evaluation, measured below), i.e., about 0.13ms per evaluation in total, approaching the cost of the base GC-OPRF. We stress that the server-side proving cost is paid offline and is the price of worst-case verifiability with public verifiability and round-optimality; sampling-based alternatives cannot achieve negligible soundness with constant amortized overhead.

For the online OT phase, we measure the malicious Soft-SpokenOT extension [52] from libOTe (the same OT extension layer as in our base implementation) with 128-bit messages given offline base OTs: 32,768 OTs (a batch of $m = 128$ evaluations with the probe-resilient encoding at $n = 2\lambda$) take 5.6ms and 531KB in total, i.e., 0.044ms and 4.1KB per evaluation.

Comparison with related work. Table VII compares the online performance of our VOPRF with related (V)OPRFs, focusing on the online phase. Our numbers are all measured on the Apple M5 Pro; the numbers of [61] are measured on an AWS m5.large machine in their paper, and the numbers of [26] are estimates without an implementation.

Several observations are in order. First, our online phase is two-round, matching the round optimum; Gold requires five rounds (three with Fiat-Shamir) and 2Hash requires nine. Second, our amortization already takes effect at a batch size of $m = 128$, whereas Gold’s stated amortized costs require $n \approx 3 \times 10^5$ evaluations (at $n = 10^3$, its costs degrade to 17.1KB and 24.4ms per evaluation); for unbalanced PSI applications such as password breach checking, where the client holds only a few items, our operating point is the more suitable one. Third, our VOPRF is *publicly* verifiable: anyone holding the key commitment can verify the batch proof, whereas the verification of Gold and LeOPaRd is designated to a client holding secret state (VOLE-based ZK and LaBRADOR respectively). We note that Gold’s public verification key (a few PRF values at public points) is conceptually similar to our key commitment; the difference is that our commitment comes with a STARK proof of key knowledge (enabling extraction in the security proof) and that our per-batch proof verifies the evaluations themselves. Fourth, our assumptions

TABLE V. ONLINE PERFORMANCE COMPARISON OF (V)OPRFs. *: CLASSICAL (NOT POST-QUANTUM), SHOWN AS A REFERENCE POINT. †: MEASURED BY THE AUTHORS ON AN AWS M5.LARGE; BATCHED FIGURES REQUIRE $n \approx 3 \times 10^5$ FOR THE STATED AMORTIZED COST (AT $n = 10^3$: 17.1KB AND 24.4MS). ‡: ESTIMATES WITHOUT IMPLEMENTATION. OUR ROWS ARE MEASURED ON AN APPLE M5 PRO (BATCH $m = 128$); "ONLINE COMM." EXCLUDES THE GARBLED TABLES AND THE PROOF, WHICH ARE INPUT-INDEPENDENT AND SENT OFFLINE.

Work	Online rounds	Online comm./eval	Online client/eval	Online server/eval	Verifiability	Soundness
2HashDH-VOPRF [23]*	2	$\approx 0.1\text{KB}$	$< 1\text{ms}$	$< 1\text{ms}$	public	negligible
Gold [61]†	5 (3 w/ FS)	1.9KB	1ms	1ms	designated	negligible
LeOPaRd [26]‡	2	$\approx 13\text{KB}$	—	—	designated	negligible
2Hash [10]	9	$\approx 911\text{KB}$	570ms (E2E)		UC	negligible
Leap [34]	2	23KB	0.011ms	0.75ms	none	—
Ours (tables offline)	2	$\approx 5\text{KB}$	0.13ms	$\approx 0.1\text{ms}$	public	negligible
Ours (tables online)	2	$\approx 210\text{KB}$	0.13ms	$\approx 0.1\text{ms}$	public	negligible

Note that Gold's security holds against quantum adversaries with classical queries only, and that Leap is semi-honest secure and not verifiable.

are conservative: AES as a PRF and any IND-CPA secure KEM (for the OT), with no restriction to classical queries (in contrast to the power-residue PRF underlying Gold) and no new interactive assumptions (in contrast to LeOPaRd). On the negative side, if the garbled tables must be delivered online, our online communication is about 0.2MB per evaluation, which is larger than Gold's and Leap's; and our server-side proving cost (about 5.2s per circuit) is substantial, albeit fully offline, input-independent, and parallelizable.

VIII. CONCLUSION

In this paper, we propose a practical two-round OPRF with malicious security based on lattice-based assumptions and symmetric key primitives that are believed to be post-quantum secure. Our OPRF model does not require verifiability, hence the construction can be designed using a malicious-secure OT protocol from any IND-CPA secure key encapsulation mechanism and a semi-honest secure Garbled Circuit with high efficiency. We further upgrade the OPRF to a batch verifiable OPRF with post-quantum security, in which a one-time, reusable, and publicly verifiable key commitment binds the server to a single AES key, and a per-batch STARK proof of correct garbling guarantees that every accepted evaluation is consistent with the committed key; the proof covers only the garbling step and is fully precomputable offline, so the online phase remains two-round with client cost approaching that of the base OPRF. Furthermore, we utilize our proposed practical OPRF to design efficient PPSS and E2SE protocols, and our VOPRF to design a post-quantum verifiable PSI protocol in the unbalanced setting with password breach checking as a typical application. Our work contributes to the development of post-quantum secure cryptographic primitives, which are essential for securing sensitive information in the quantum computing era.

REFERENCES

- [1] J. Abascal, M. H. F. Sereshgi, C. Hazay, Y. Ishai, and M. Venkatasubramanian, "Is the classical GMW paradigm practical? the case of non-interactive actively secure 2PC," in *CCS*. ACM, 2020, pp. 1591–1605.
- [2] M. R. Albrecht, A. Davidson, A. Deo, and D. Gardham, "Crypto dark matter on the torus: Oblivious PRFs from shallow PRFs and FHE," *IACR Cryptol. ePrint Arch.*, p. 232, 2023.
- [3] M. R. Albrecht, A. Davidson, A. Deo, and N. P. Smart, "Round-optimal verifiable oblivious pseudorandom functions from ideal lattices," in *Public Key Cryptography (2)*, ser. Lecture Notes in Computer Science, vol. 12711. Springer, 2021, pp. 261–289.
- [4] A. Bagherzandi, S. Jarecki, N. Saxena, and Y. Lu, "Password-protected secret sharing," in *CCS*. ACM, 2011, pp. 433–444.
- [5] A. Banerjee, C. Peikert, and A. Rosen, "Pseudorandom functions and lattices," in *EUROCRYPT*, ser. Lecture Notes in Computer Science, vol. 7237. Springer, 2012, pp. 719–737.
- [6] A. Basso, "A post-quantum round-optimal oblivious PRF from isogenies," *Cryptology ePrint Archive*, 2023.
- [7] A. Basso, P. Kutas, S.-P. Merz, C. Petit, and A. Sanso, "Cryptanalysis of an oblivious PRF from supersingular isogenies," in *Advances in Cryptology—ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part I* 27. Springer, 2021, pp. 160–184.
- [8] C. Baum, L. Braun, A. Munch-Hansen, and P. Scholl, "Moz z 2 k arella: efficient vector-ole and zero-knowledge proofs over z 2 k," in *Advances in Cryptology—CRYPTO 2022: 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15–18, 2022, Proceedings, Part IV*. Springer, 2022, pp. 329–358.
- [9] E. Ben-Sasson, I. Bentov, Y. Horeish, and M. Riabzev, "Scalable, transparent, and post-quantum secure computational integrity," *Cryptology ePrint Archive*, Paper 2018/046, 2018.
- [10] W. Beullens, L. Dodgson, S. Faller, and J. Hesse, "The 2Hash OPRF framework and efficient post-quantum instantiations," in *EUROCRYPT*, ser. Lecture Notes in Computer Science, vol. 15608. Springer, 2025, pp. 332–362.
- [11] W. Beullens and G. Seiler, "Labrador: Compact proofs for r1cs from module-sis," *Cryptology ePrint Archive*, 2022.
- [12] D. Boneh, Y. Ishai, A. Passelègue, A. Sahai, and D. J. Wu, "Exploring crypto dark matter: New simple PRF candidates and their applications," in *Theory of Cryptography: 16th International Conference, TCC 2018, Panaji, India, November 11–14, 2018, Proceedings, Part II*. Springer, 2018, pp. 699–729.
- [13] D. Boneh, D. Kogan, and K. Woo, "Oblivious pseudorandom functions from isogenies," in *ASIACRYPT (2)*, ser. Lecture Notes in Computer Science, vol. 12492. Springer, 2020, pp. 520–550.
- [14] E. Boyle, G. Couteau, N. Gilboa, and Y. Ishai, "Compressing vector OLE," in *CCS*. ACM, 2018, pp. 896–912.
- [15] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, N. Resch, and P. Scholl, "Correlated pseudorandomness from expand-accumulate codes," in *Advances in Cryptology—CRYPTO 2022: 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15–18, 2022, Proceedings, Part II*. Springer, 2022, pp. 603–633.
- [16] R. Canetti, U. Feige, O. Goldreich, and M. Naor, "Adaptively secure multi-party computation," in *STOC*. ACM, 1996, pp. 639–648.
- [17] S. Casacuberta, J. Hesse, and A. Lehmann, "Sok: Oblivious pseudorandom functions," in *IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2022, pp. 625–646.
- [18] N. Chandran, D. Gupta, and A. Shah, "Circuit-PSI with linear complexity via relaxed batch OPPRF," *Proc. Priv. Enhancing Technol.*, vol. 2022, no. 1, pp. 353–372, 2022.
- [19] D. R. Chawdhuri and M. Prabhakaran, "Secret-carrying puzzles and

- garbled circuits optimized for zero-knowledge proofs,” Cryptology ePrint Archive, Paper 2026/788, 2026.
- [20] L. Chen, Y. Li, Q. Tang, and M. Yung, “End-to-same-end encryption: Modularly augmenting an App with an efficient, portable, and blind cloud storage,” in *USENIX Security Symposium*. USENIX Association, 2022, pp. 2353–2370.
 - [21] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “TFHE: fast fully homomorphic encryption over the torus,” *J. Cryptol.*, vol. 33, no. 1, pp. 34–91, 2020.
 - [22] D. Dachman-Soled, J. Katz, and V. Rao, “Adaptively secure, universally composable, multiparty computation in constant rounds,” in *TCC (2)*, ser. Lecture Notes in Computer Science, vol. 9015. Springer, 2015, pp. 586–613.
 - [23] A. Davidson, A. Faz-Hernandez, N. Sullivan, and C. A. Wood, “Oblivious pseudorandom functions (OPRFs) using prime-order groups,” IETF, Tech. Rep. RFC 9497, 2024.
 - [24] B. E. Diamond and J. Posen, “Binus: a hardware-optimized SNARK over binary tower fields,” Cryptology ePrint Archive, Paper 2023/1784, 2023.
 - [25] I. Dinur, S. Goldfeder, T. Halevi, Y. Ishai, M. Kelkar, V. Sharma, and G. Zaverucha, “MPC-friendly symmetric cryptography from alternating moduli: candidates, protocols, and applications,” in *Advances in Cryptology—CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part IV 41*. Springer, 2021, pp. 517–547.
 - [26] M. F. Esgin, R. Steinfeld, E. Tairi, and J. Xu, “Leopard: Towards practical post-quantum oblivious PRFs via interactive lattice problems,” Cryptology ePrint Archive, Paper 2024/1615, 2024.
 - [27] A. Everspaugh, R. Chatterjee, S. Scott, A. Juels, and T. Ristenpart, “The Pythia PRF service,” in *USENIX Security Symposium*. USENIX Association, 2015, pp. 547–562.
 - [28] S. H. Faller, A. Ottenhues, and J. Ottenhues, “Composable oblivious pseudo-random functions via garbled circuits,” in *LATINCRYPT*, ser. Lecture Notes in Computer Science, vol. 14168. Springer, 2023, pp. 249–270.
 - [29] M. J. Freedman, Y. Ishai, B. Pinkas, and O. Reingold, “Keyword search and oblivious pseudorandom functions,” in *TCC*, ser. Lecture Notes in Computer Science, vol. 3378. Springer, 2005, pp. 303–324.
 - [30] O. Goldreich, S. Goldwasser, and S. Micali, “How to construct random functions,” in *Providing Sound Foundations for Cryptography*. ACM, 2019, pp. 241–264.
 - [31] C. Guo, J. Katz, X. Wang, C. Weng, and Y. Yu, “Better concrete security for half-gates garbling (in the multi-instance setting),” in *CRYPTO (2)*, ser. Lecture Notes in Computer Science, vol. 12171. Springer, 2020, pp. 793–822.
 - [32] U. Haböck, “Multivariate lookups based on logarithmic derivatives,” Cryptology ePrint Archive, Paper 2022/1530, 2022.
 - [33] C. Hazay and Y. Lindell, “Efficient protocols for set intersection and pattern matching with security against malicious and covert adversaries,” *J. Cryptol.*, vol. 23, no. 3, pp. 422–456, 2010.
 - [34] L. Heimberger, D. Kales, R. Lolato, O. Mir, S. Ramacher, and C. Rechberger, “Leap: A fast, lattice-based OPRF with application to private set intersection,” in *EUROCRYPT*, ser. Lecture Notes in Computer Science. Springer, 2025.
 - [35] L. Heimberger, F. Meisinger, and C. Rechberger, “OPRFs from isogenies: Designs and analysis,” *Cryptology ePrint Archive*, 2023.
 - [36] S. Jarecki, A. Kiayias, and H. Krawczyk, “Round-optimal password-protected secret sharing and T-PAKE in the password-only model,” in *ASIACRYPT (2)*, ser. Lecture Notes in Computer Science, vol. 8874. Springer, 2014, pp. 233–253.
 - [37] S. Jarecki, A. Kiayias, H. Krawczyk, and J. Xu, “Highly-efficient and composable password-protected secret sharing (or: How to protect your bitcoin wallet online),” in *EuroS&P*. IEEE, 2016, pp. 276–291.
 - [38] S. Jarecki, H. Krawczyk, and J. K. Resch, “Updatable oblivious key management for storage systems,” in *CCS*. ACM, 2019, pp. 379–393.
 - [39] S. Jarecki, H. Krawczyk, and J. Xu, “OPAQUE: an asymmetric PAKE protocol secure against pre-computation attacks,” in *EUROCRYPT (3)*, ser. Lecture Notes in Computer Science, vol. 10822. Springer, 2018, pp. 456–486.
 - [40] M. Keller, “MP-SPDZ: A versatile framework for multi-party computation,” in *CCS*. ACM, 2020, pp. 1575–1590.
 - [41] V. Kolesnikov, R. Kumaresan, M. Rosulek, and N. Trieu, “Efficient batched oblivious PRF with applications to private set intersection,” in *CCS*. ACM, 2016, pp. 818–829.
 - [42] R. W. F. Lai, C. Egger, M. Reinert, S. S. M. Chow, M. Maffei, and D. Schröder, “Simple password-hardened encryption services,” in *USENIX Security Symposium*. USENIX Association, 2018, pp. 1405–1421.
 - [43] R. W. F. Lai, C. Egger, D. Schröder, and S. S. M. Chow, “Phoenix: Rebirth of a cryptographic password-hardening service,” in *USENIX Security Symposium*. USENIX Association, 2017, pp. 899–916.
 - [44] Y. Lindell and B. Pinkas, “An efficient protocol for secure two-party computation in the presence of malicious adversaries,” in *EUROCRYPT*, ser. Lecture Notes in Computer Science, vol. 4515. Springer, 2007, pp. 52–78.
 - [45] D. Masny and P. Rindal, “Endemic oblivious transfer,” in *CCS*. ACM, 2019, pp. 309–326.
 - [46] M. Naor and O. Reingold, “Number-theoretic constructions of efficient pseudo-random functions,” in *FOCS*. IEEE Computer Society, 1997, pp. 458–467.
 - [47] J. B. Nielsen, P. S. Nordholt, C. Orlandi, and S. S. Burra, “A new approach to practical active-secure two-party computation,” in *CRYPTO*, ser. Lecture Notes in Computer Science, vol. 7417. Springer, 2012, pp. 681–700.
 - [48] B. Pinkas, T. Schneider, O. Tkachenko, and A. Yanai, “Efficient circuit-based PSI with linear communication,” in *EUROCRYPT (3)*, ser. Lecture Notes in Computer Science, vol. 11478. Springer, 2019, pp. 122–153.
 - [49] B. Pinkas, T. Schneider, and M. Zohner, “Scalable private set intersection based on OT extension,” *ACM Trans. Priv. Secur.*, vol. 21, no. 2, pp. 7:1–7:35, 2018.
 - [50] M. O. Rabin, “How to exchange secrets with oblivious transfer,” *IACR Cryptol. ePrint Arch.*, p. 187, 2005.
 - [51] P. Rindal and P. Schoppmann, “VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE,” in *EUROCRYPT (2)*, ser. Lecture Notes in Computer Science, vol. 12697. Springer, 2021, pp. 901–930.
 - [52] L. Roy, “SoftspokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model,” in *CRYPTO (1)*, ser. Lecture Notes in Computer Science, vol. 13507. Springer, 2022, pp. 657–687.
 - [53] J. Schneider, N. Fleischhacker, D. Schröder, and M. Backes, “Efficient cryptographic password hardening services from partially oblivious commitments,” in *CCS*. ACM, 2016, pp. 1192–1203.
 - [54] I. A. Seres, M. Horváth, and P. Bursi, “The legendre pseudorandom function as a multivariate quadratic cryptosystem: Security and applications,” *IACR Cryptol. ePrint Arch.*, p. 182, 2021.
 - [55] M. Shirvanian, S. Jarecki, H. Krawczyk, and N. Saxena, “SPHINX: A password store that perfectly hides passwords from itself,” *IACR Cryptol. ePrint Arch.*, p. 695, 2018.
 - [56] N. Tyagi, S. Celi, T. Ristenpart, N. Sullivan, S. Tessaro, and C. A. Wood, “A fast and simple partially oblivious PRF, with applications,” in *Advances in Cryptology—EUROCRYPT 2022: 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30–June 3, 2022, Proceedings, Part II*. Springer, 2022, pp. 674–705.
 - [57] D. Unruh, “Non-interactive zero-knowledge proofs in the quantum random oracle model,” in *EUROCRYPT (2)*, ser. Lecture Notes in Computer Science, vol. 9057. Springer, 2015, pp. 755–784.
 - [58] X. Wang, S. Ranellucci, and J. Katz, “Authenticated garbling and efficient maliciously secure two-party computation,” in *CCS*. ACM, 2017, pp. 21–37.
 - [59] K. Yang, P. Sarkar, C. Weng, and X. Wang, “Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2986–3001.
 - [60] R. Yang, M. H. Au, Z. Zhang, Q. Xu, Z. Yu, and W. Whyte, “Efficient lattice-based zero-knowledge arguments with standard soundness: Construction and applications,” in *CRYPTO (1)*, ser. Lecture Notes in Computer Science, vol. 11692. Springer, 2019, pp. 147–175.

- [61] Y. Yang, F. Benhamouda, S. Halevi, H. Krawczyk, and T. Rabin, "Gold OPRF: Post-quantum oblivious power-residue PRF," in *IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2025.
- [62] A. C. Yao, "How to generate and exchange secrets (extended abstract)," in *FOCS*. IEEE Computer Society, 1986, pp. 162–167.
- [63] S. Zahur, M. Rosulek, and D. Evans, "Two halves make a whole - reducing data transfer in garbled circuits using half gates," in *EUROCRYPT* (2), ser. Lecture Notes in Computer Science, vol. 9057. Springer, 2015, pp. 220–250.

APPENDIX A APPLICATION IN E2SE

A. End to Same End Encryption

The system of End to Same End Encryption (E2SE) [20] is designed for storing private data in the cloud with the help of a key server. It aims to achieve four goals simultaneously: portability, cloud-blindness, deployability, and built-in App login.

Portability: E2SE enables users to securely access their content and login to the App on any device using a single passphrase. This ensures that the system remains portable and allows users to access their data seamlessly across different devices. No secret information is stored on the user's device, enhancing the security of the system.

Cloud-blindness: E2SE provides "end-to-same-end" authenticated encryption for data. It ensures that the data remains secure even in the presence of a compromised server and protects against illegitimate users who do not follow the protocol. Neither server can manipulate the client into accepting tampered data, ensuring the integrity and confidentiality of the stored information.

Deployability on plain cloud storage: E2SE minimizes the requirements for the data server, making it compatible with existing plain storage services such as Amazon S3 or Dropbox. The data server only needs to support password login and basic data storage/retrieval functions, allowing for easy deployment without significant infrastructure changes.

Built-in App login: E2SE integrates the App login module, eliminating the need for users to enter separate passphrases for App login and the secure storage module. This simplifies the user experience and reduces the risk of security issues arising from careless passphrase management.

We provide a brief overview of the structure of the E2SE system in Figure 13. For detailed definitions and construction of the system, refer to [20] for further explanation.

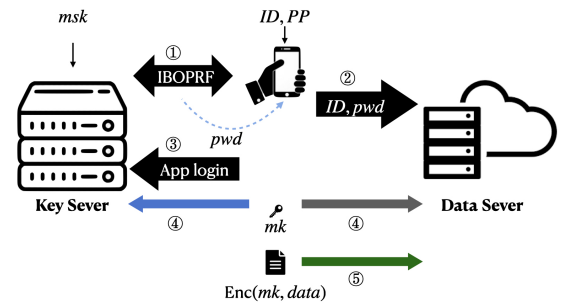


Fig. 13. E2SE data flow. 1). the client derives his password by querying the key server with his passphrase. 2). the client logs in the data server with his ID and the derived password. 3). the client logs in the key server. 4). the client "shares" his master key. 5). the client encrypts his data with the master key and deposit the ciphertext to the data server.

B. Identity Based Oblivious PRF

The identity based oblivious pseudorandom function (IBOPRF) is an important primitive underlying E2SE to insist on a single password. We adopt the definition of IBOPRF as provided in [20].

1. $\text{IBSetup}(1^\lambda)$: $pp \leftarrow \text{Setup}(1^\lambda)$, $msk \leftarrow \text{KeyGen}(pp)$. Output (pp, msk) .
2. $\text{IBCEval}_1(pp, id, x)$: $(st, ch) \leftarrow \text{CEval}_1(pp, x^*)$. Output (st, ch) .
3. $\text{IBSEval}(pp, id, ch, msk)$: $k_{id} := H_3(id, msk)$. $rp \leftarrow \text{SEval}(pp, ch, k_{id})$. Output rp . // Server generates a client-specific PRF key with a random oracle.
4. $\text{IBCEval}_2(pp, id, rp, st)$: $y \leftarrow \text{CEval}_2(pp, rp, st)$. Output y .

Fig. 14. The construction of IBOPRF from OPRF.

Definition 14. The IBOPRF is a challenge-response protocol between a client and a server with the following syntax.

1. $\text{IBSetup}(1^\lambda)$: Given the security parameter λ , generate the public parameter pp and the server's master secret key msk .
2. $\text{IBCEval}_1(pp, id, x)$: Given the public parameter pp , identity id and a secret input passphrase x , the client computes a challenge ch and an internal state st .
3. $\text{IBSEval}(pp, id, ch, msk)$: When the server receives an identity id and a challenge ch from the client, the server computes a response rp according to the public parameter pp and his master secret key msk .
4. $\text{IBCEval}_2(pp, id, rp, st)$: When the client receives the response rp from the server, he will retrieve the internal state st and compute the function output password y according to the public parameter pp and the identity id .

The IBOPRF should guarantee the following properties (Formal definitions are similar with ours in OPRF):

Uniqueness. If all parties follow the protocol, the client will learn a unique output $y = \mathcal{F}_k(id, x)$, i.e., the client will never output $y' \neq y$ for same id and x .

Pseudorandomness. Intuitively, pseudorandomness captures the security of IBOPRF against other malicious clients who did not corrupt the server but can arbitrarily query the server. It guarantees that those clients cannot distinguish $y = \mathcal{F}_k(id^*, x^*)$ from a random string r for a chosen identity id^* and a secret input x , even though he can arbitrarily query the server. More precisely, we have the following definition.

Obliviousness. It models the security of IBOPRF against the malicious server. Obliviousness guarantees that a malicious server cannot predict y for a fixed id and an unknown input x , even if it can interact with the honest client holding id and x multiple times.

C. Construction

The construction of E2SE is built upon symmetric primitives that are resistant to quantum attacks, with the exception of the IBOPRF. To address this limitation, we replace the 2HashDH-based IBOPRF with a lattice-based IBOPRF, resulting in a lattice-based version of E2SE.

Intuitively, the key distinction between OPRF and IBOPRF lies in the fact that the output of an IBOPRF should correspond to a specific public identity id . By combining the public identity id and the master secret key msk in a random oracle, we can generate a client-specific pseudo-random function (PRF) key and then get an IBOPRF with an OPRF. The construction is shown as Figure. 14. The proof is trivial so we omit here.